## Table of Contents

**Chapter 1** ..... **3**

**Introduction** ..... **3**

*1.1 Background* ..... *3*

*1.2 Objective* ..... *4*

*1.3 Purpose, Scope and Applicability* ..... *5*

1.3.1 Purpose ..... 5

1.3.2 Scope ..... 6

1.3.3 Applicability ..... 6

*1.4 Achievements* ..... *7*

*1.5 Organization of Report* ..... *7*

**Chapter 2** ..... **8**

**Survey of Technologies** ..... **8**

**Chapter 3** ..... **12**

**Requirements and Analysis** ..... **12**

*3.1 Problem Definition* ..... *12*

*3.2 Requirements Specification* ..... *13*

*3.3 Planning and Scheduling* ..... *15*

*3.4 Software and Hardware Requirements* ..... *16*

*3.5 Preliminary Product Description* ..... *17*

**Chapter 4** ..... **19**

**System Design** ..... **19**

*4.1 Basic Modules* ..... *19*

USE CASE SCENARIOS ..... 21

*4.2 Data Design* ..... *37*

4.2.1 Schema Design ..... 38

4.2.2 Data Integrity and Constraints ..... 39

*4.3 Procedural Design* ..... *39*

4.3.1 Logic Diagrams ..... 39

*4.4 User interface design* ..... *48*

*4.5 Security Issues* ..... *51*

*4.6 Test Cases Design* ..... *52*

**Chapter 5** ..... **59**

**Implementation and Testing** ..... **59**

*5.1 Implementation Approaches* ..... *59*

*5.2 Coding Details and Code Efficiency* ..... *60*

5.2.1 Code Efficiency ..... 60

*5.3 Testing Approach* ..... *71*

5.3.1 Unit Testing ..... 71

5.3.2 Integrated Testing ..... 71

*5.4 Modifications and Improvements* ..... *72*

*5.5 Test Cases* ..... *73*

**Chapter 6** ..... **75**

**Results and Discussion** ..... **75**

*6.1 Test Reports* ..... *75*

*6.2 User Documentation* ..... *78*

**Chapter 7** ..... **84**

**Conclusions** ..... **84**

*7.1 Conclusion* ..... *84*

*7.2 Limitations of the system* ..... *84*

*7.3 Future Scope of the Project* ..... *85*

# Chapter 1

# Introduction

## 1.1 Background

After BMC's e-governance mobile application MCGM 24x7 fails pothole test, it's website mcgm.gov.in fares no better.

**Potholes are the lowly annoyance that in fact cause billions of dollars in vehicle damages and a large portion of highway deaths.**

The glitch is city has nowhere to report bad roads. Also earlier, government worked on the complaint for potholes, but not all demands of the public were fulfilled. Potholes and badly designed speed breakers don't just irritate drivers and damage vehicles. They actually kill.

Talking about out of Mumbai, Maharashtra- For the first time, the government recorded deaths caused by potholes, speed breakers and humps on roads. Last year, about 11,400 people died in such cases with Uttar Pradesh topping the list.

Other states with a high number of fatalities on these counts included Madhya Pradesh, Tamil Nadu, Karnataka and West Bengal, Maharashtra registered 368 deaths in crashes due to potholes while another 224 persons died in crashes on roads that were under repair or under construction.

Potholes occur due to wear and tear of roads especially in the monsoon, resulting in accumulating rainwater.

## 1.2 Objective

The BMC's app where citizens can submit complaints about potholes and other civic issues is as frustrating as the roads in the civic body's jurisdiction.

To ensure that majority of potholes on roads/highways are resolved by communication/interaction between public & government on the system/website.

Some key objectives include-

- To classify the road damage for reporting purposes.
- To provide an image of the damage.
- To provide an accurate location of the road damage.
- To map data so that it is easily understood.
- To provide a graph that will show users an overall view of system working on pothole complaints.
- Simple and Uncomplicated User Interface.
- Notification about updated status.

## 1.3 Purpose, Scope and Applicability

### 1.3.1 Purpose

A pothole is a structural failure in a road surface, usually asphalt pavement due to water in the underlying soil structure and traffic passing over the affected area.

The service is in addition to BMC's mobile application MCGM24X7 that allows citizens to geo-tag a particular road patch and file complaints. This app doesn't allow exclusive complaints on potholes, but accepts complaints about bad patches.

There should be some system which is not only confined to websites. There are difficulties relating to websites of all municipal authorities. A simple mechanism should be provided so that a person can click a photo and upload it on a website or lodge a complaint through mobile phone. And the most important point that the government must maintain a list of complaints and reply within a specified time.

As and now due to increasing potholes vehicles cannot be driven properly, which directly leads to increase in traffic.

In short to reduce or overcome potholes problems, Pothole Complaint System is required.

### 1.3.2 Scope

Public facing roads and traffic problems can easily register their respective complaint of potholes on the website application using Mobile Phones/Tablets.

Main issues covered in the project-As there are several problems in the old applications like the app freezes in the middle of filling a complaint, raised many follow-up complaints but still no action, etc.

Functions of the project-This website application can be modified according to admin as well as for Users with better UI. Email Notification is the easiest and most interactive way for public to get information about the process of actions.

This System can be used in a particular area managed by the respective Admin/Council.

Issue- Users don't get to know whether working on their complaint has started or not. Or what is the working status.

Solution- Users can see the actual status of our complaint i.e. the current complaint working status

### 1.3.3 Applicability

The website application is applicable firstly in those areas which are underdeveloped or in the developing process.

No doubt, even in developed/urban areas there are still roads and highways which are not yet properly constructed or some are damaged after construction; Areas like these have applicability of this System.

Potholes lead to accidents as well as pollution due to the flying dust. With the web application, we will know the exact status of roads and the agency responsible.

People will have a benefit of registering a complaint & having a smooth interaction with government throughout the process.

## 1.4 Achievements

After the completion of work, knowledge gained is how a government website should process a complaint by taking actions step by step on registered complaints. Sequences of steps are performed by User, SubAdmin and MainAdmin right from a pothole complaint is registered to the pothole is reconstructed successfully.

A goal achieved is that Users can now get notification of complaint's updated status.

## 1.5 Organization of Report

In 2nd chapter-Survey of Technologies, what all technologies have been used in the project is discussed.

In 3rd chapter-Requirements and Analysis, requirements of the topic, planning and scheduling of modules creation, software and hardware required to work in within are discussed.

In 4th chapter-System Design, modules, database and table structure, UML diagrams representing relationships, UI designs, Security issues and test cases performed on each module.

In 5th chapter-Implementation and Testing, Programming part i.e. explanation of some important codes and testing approach on functions and what approach the code is for is discussed.

In 6th chapter-Results and Discussion, test reports based on test cases, how the software works and document showing all possible information about working software.

In 7th chapter-Conclusion, what are the significance, limitations, and future scope of the system and project?

# Chapter 2

# Survey of Technologies

**LIST OF TECHNOLOGIES**

***Client side Technologies:-***

1. HTML5

2. CSS3

3. Bootstrap 4

4. JavaScript

***Server side Technologies:-***

1. PHP

2. Laravel

***Database:-***

1. MySQL

## SUMMARY OF TECHNOLOGIES

***Client side Technologies:-***

HTML, CSS, and JavaScript are the core technologies used to develop the frontend of any web application.

HTML is a markup language whose elements are the building blocks of web pages.

CSS is a stylesheet language used to define styles of HTML elements.

JavaScript is a high level, interpreted programming language which enables interactive web pages and thus is an essential part of web technologies.

Bootstrap is a free and open-source front-end framework or library for developing websites and web applications. It contains HTML and CSS based design templates for various interface components like typography, form, layouts, etc.

***Server side Technologies:-***

PHP is a server side scripting language used to create dynamic and interactive web applications. It’s free and an efficient alternative to Microsoft’s ASP.

Laravel is a free, open-source PHP framework used to develop powerful web applications which follows MVC (Model-View-controller) architectural design pattern.

# OTHER ALTERNATIVES TO THE CHOSEN TECHNOLOGIES

***Alternatives to PHP / Laravel***

**1. Java / Spring MVC**

Java is a general-purpose computer-programming language that is concurrent, class-based, object-oriented, and specifically designed to have as few implementation dependencies as possible. Spring MVC is a Java framework that offers a useful toolkit for development and configuration of web applications. Spring is famous among programmers for its well-developed ecosystem.

**Why we chose Laravel over Spring :**

I. Although Spring has better security, getting started with it is quite difficult for beginners. Learning curve is not that simple as compared to Laravel.

II. Programmers need to have strong knowledge of core language i.e. Java in order to work with Spring, which is not the case with Laravel, basic knowledge of the core language i.e. PHP is sufficient.

III. It is difficult to get solutions for Spring, a lot of solutions are outdated.

IV. Laravel has Eloquent ORM (Object Relational Mapping) built-in, which is not the case in Spring, it requires other frameworks like Hibernate.

V. Laravel is very suitable for rapid development to web applications, which is indeed a very good reason for us to use it instead of Spring.

**2. ASP.NET MVC**

ASP.NET is an open-source server-side web application framework designed for web development to produce dynamic web pages. It allows programmers to use a full featured programming language such as C# or VB.NET to build web applications easily. ASP.NET MVC is a web application framework which implements the model–view–controller pattern. The components of the ASP.NET MVC framework are designed so that they can be easily replaced or customized. It reduces complexity that makes it ideal for large scale applications where multiple teams are working.

**Why we chose Laravel over ASP.NET MVC:**

I. Similar to Spring, ASP.NET's learning curve is not suitable for everyone. It is heavily based on the core language either C# or Visual Basic.

II. .NET is owned by Microsoft, whereas Laravel is free and open source.

III. .NET operates only on Microsoft Server, on the other hand Laravel can run on any web server.

# Chapter 3

# Requirements and Analysis

## 3.1 Problem Definition

Due to increasing number of potholes government has made some Website Application as well as Android Applications so that public user can report a complaint to solve the pothole problem.

The major problem faced due to pothole is accidents, traffic jams, and many others. Also some complaint reports submitted by public user are many a times not solved by government. While some Website Applications as well as some Android Applications are not much useful for submitting a pothole complaint report because government does not follow their own promises of solving the pothole issues.

Public still hope that government may come up with a new Secure, Trustable & Requirement fulfilling Website/Android Application.

## 3.2 Requirements Specification

*Functional Requirements-*

1. New Users needs to first register & create their own account.
2. User/SubAdmin/MainAdmin can login to perform their respective tasks.
3. User can submit a Pothole Complaint Report/Issue.
4. SubAdmin will update the status as the pothole reconstruction is in process.
5. MainAdmin can delete complaint.
6. User/SubAdmin/MainAdmin can view all registered complaints.
7. User can view other complaints that are from same area/ward.
8. Progress Graph to display recent progress.
9. User can submit a positive/negative suggestion/feedback of experience with their Website Application.
10. MainAdmin can assign/dismiss SubAdmin.
11. MainAdmin can delete User account.

*Non functional requirements-*

Non functional requirements define system attributes such as reliability, accuracy, maintainability and usability.

1. Reliability - The system must be reliable.
2. Accuracy - More the system short & easy to use, the more it will be accurate.
3. Portability - System needs to be portable and platform independent , i.e. it must be available for the users who use different Operating Systems.
4. Maintainability - The codes programmed in project should be easy to maintain and should also be easy to modify. The code needs to be flexible to make it maintainable.
5. Usability - The easier the software performs the task, the better is the usability.
6. Performance - How efficiently and smoothly the code runs, the better is the performance
7. Scalability - The capability of software is basically meant by scalability. According to the technologies used the capability of the software can be scalable.

## 3.3 Planning and Scheduling

**Planning-**

Planning can be thought as determining all the small tasks that must be carried out in order to accomplish the goal. Which task should be done how & when, comes under Planning. Planning also takes into account, rules, known as constraints, which control when certain tasks can or cannot happen.

Planning of the project activities should be done before starting a project, not only to complete all the tasks, but also to get an idea that in how much time a project activity can be carried out.

**Scheduling-**

Scheduling can be thought as determining whether adequate resources are available to carry out the plan. For how much time a function should be given time so that it can be created properly.

A project with no starting and finishing date is less likely to succeed. Objective of developing our project schedule are:

I. To identify project activities that needs to be completed.
II. To estimate duration of project activities.
III. To determine the sequence and time when the project activities will need to happen.
IV. To monitor and control the project schedule.

## 3.4 Software and Hardware Requirements

**Software Requirements-**

Our platform would only be accessible through a web browser.

Following modern browsers would support our platform:

1. Google Chrome
2. Internet Explorer
3. Mozilla Firefox
4. Opera

**Hardware Requirements-**

1. RAM - 8 GB.
2. System type - 64-bit Operating System.
3. Processor – Intel(R) Core(TM) i7-6600U CPU @ 2.60GHz  2.80GHz.
4. Windows 10.

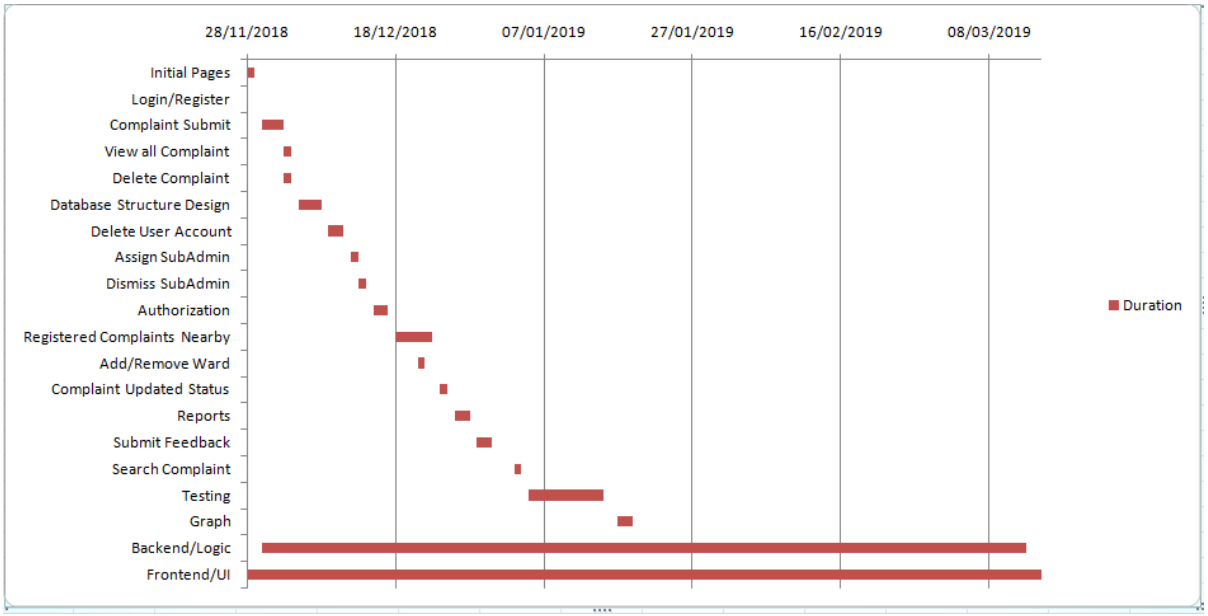
## 3.5 Preliminary Product Description

As discussed earlier, the main requirement is that the User and SubAdmin should be in touch regarding complaint's status. The only requirement is to fulfil user's expectation after submitting a complaint.

| Function | Operation |
|---|---|
| Login/Register | All 3 actors can perform this operation. |
| Submit Complaint | User submits a pothole complaint. |
| View all Complaint | All 3 actors can view all registered complaints till the date. |
| Delete Complaint | MainAdmin can delete any of complaint. |
| Delete User Account | MainAdmin can delete a user account if necessary. |
| Assign/Dismiss SubAdmin | MainAdmin assigns/dismisses a SubAdmin to/from a Ward. |
| Registered Complaints Nearby | User can view pothole complaints registered from his/her area/ward. |
| Add/Remove Ward | MainAdmin can add or remove ward according to services allowed to be served. |
| Reports | A report will show two categories i.e. Solved and Unsolved Complaints to all 3 actors. |
| Submit Feedback/Add Suggestion | User will submit a final feedback of how they used the services provided to them. |
| Search Complaint | SubAdmin & MainAdmin can search complaint to take any action on them. |
| Graph | Reports, Ratings and Complaint Status graphs are displayed to all 3 actors to have an idea of what's the status and progress of the website. |

## Gantt Chart

A Gantt Chart shows how much time was required for development of modules in terms of days/weeks/months which depends on User.



# Chapter 4

# System Design

## 4.1 Basic Modules

Here the overall system will be divided into small modules and will be made ready to be executed one by one. These modules will be further implemented together to make a whole project.

The main objective to divide the overall components into small modules is to manage each of the parts and develop each part or module separately. After successful development of each module we will/can integrate all the modules into one system.

Some important modules that are divided from the system to develop it separately are:-

1. Submit/Register Complaint.

   User registers a pothole complaint.

2. Graph of Status/Progress.

   User, SubAdmin, MainAdmin views the graph to see the solved/unsolved complaints, ratings and number of complaints having different status.

3. View all complaints.

   All 3 actors can view all registered complaints till date.

4. Add suggestions.

   User can suggest a positive/negative feedback regarding the work, system, etc by submitting a suggestion to the portal.

5. Delete User Accounts.

   MainAdmin can delete some accounts that are fake/not registered any complaint/duplicate account or another case.

6. Assign/Dismiss SubAdmin.

   MainAdmin can assign or dismiss SubAdmin according to area requirements or non-requirements respectively.

7. Search Complaint.

   MainAdmin & SubAdmin both can search any complaint by taking any 1 attribute under consideration and take respective action.

8. Reports.

   A sample result of some data by filtering needed attributes can be generated by using this function.

# USE CASE SCENARIOS

*UCF01-SignUp/Registration*

*UCF02-Login*

*UCF03-Submit/Register Complaint*

*UCF04-Update Status*

*UCF05-Submit Feedback/Add Suggestion*

*UCF06-Registered Complaints Nearby*

*UCF07-View All Complaint*

*UCF08-Delete Complaint*

*UCF09- Graph of Status/Progress*

*UCF10-Assign/Dismiss SubAdmin*

*UCF11-Delete User Account*

*UCF12-Add/Remove Ward*

*UCF13-Reports*

*UCF14-Search Complaint*

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F01 | | |
| Use Case Name: | SignUp/Registration | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 15-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | User |
| Description: | First step for a new user to use the system. |
| Preconditions: | 1. User should submit correct & relevant details to create account.<br>2. Must have an EmailID for authentication. |
| Postconditions: | 1. User should remember their username & password to login. |
| Priority: | High, Because a user must be first registered to login. |
| Frequency of Use: | This function can be executed as and when user wants to create their account. |
| Normal Course of Events: | 1. User goes to signup page.<br>2. Enters and submits info/details. |
| Exceptions: | F01.EX.1-User enters invalid EmailID. |
| Includes: | Executed by User. |
| Special Requirements: | On entering wrong details system can suggest an example of a field to enter. |
| Notes and Issues: | Signing Up is the first and main process to execute rest of the functionality. All details submitted must be correct. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F02 | | |
| Use Case Name: | Login | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 15-09-2018 | Date last updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | User, SubAdmin, MainAdmin |
| Description: | User/SubAdmin/MainAdmin will login to perform their respective tasks. |
| Preconditions: | For user, username & password should be same as entered during signup process.<br>SubAdmin & MainAdmin will have their own Username & Password. |
| Priority: | High |
| Frequency of Use: | User/SubAdmin/MainAdmin can login any number of time as they need. |
| Normal Course of Events: | 1. User/SubAdmin/MainAdmin enters username & password.<br>2. If both fields match with record in database then gets redirected to webpage. |
| Alternative Courses: | F02.AC.1- User/SubAdmin/MainAdmin logins & immediately logouts from their profile. |
| Exceptions: | F02.EX.1-Connection to database is lost. |
| Includes: | A necessary function to be executed before performing any task on portal related to system. |
| Assumptions: | System expects to accept correct Username & Password by User/SubAdmin/MainAdmin. |
| Notes and Issues: | Mandatory function that needs to be executed by all 3 actors for entering into portal. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F03 | | |
| Use Case Name: | Submit/Register Complaint | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 29-08-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | User |
| Description: | All the steps are performed by the user. |
| Preconditions: | 1. User must be logged in.<br>2. User must give each and every detail of the pothole in order to submit the complaint. |
| Postconditions: | 1. Once a step is performed user cannot undo it or cannot repeat a previous step. |
| Priority: | High, depends how user interacts in the process. |
| Frequency of Use: | Performed only once for a user. |
| Normal Course of Events: | 1. User saw a pothole.<br>2. Registers & Logins on the portal.<br>3. Fills all required details.<br>4. Submits the complaint. |
| Alternative Courses: | F023AC.1-User may never focus on updated status of complaint. |
| Exceptions: | F03.EX.1-User is unable to submit any detail or the complaint. |
| Includes: | Whole scenario is performed by the user step by step sequentially. |
| Assumptions: | SubAdmin should assume that the user will submit following as well as each & every detail of complaint. |
| Notes and Issues: | 1. If any of detail about complaint is missing, system will not accept the complaint. |

| | |
|---|---|
| | 2. Complaint once submitted cannot be edited. It can only be deleted by MainAdmin. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F04 | | |
| Use Case Name: | Update Status | | |
| Created By: | Vicky Gupta | Last Updated By: | |
| Date Created: | 13-03-2019 | Date Last Updated: | |

| | |
|---|---|
| Actor: | SubAdmin |
| Description: | SubAdmin will view the complaints registered under his ward and update complaint status according to work performed. |
| Preconditions: | 1. SubAdmin must be authorized & logged in.<br>2. SubAdmin should know which complaint to be assigned with which complaint. |
| Postconditions: | 1. SubAdmin cannot assign an old status to a complaint. |
| Priority: | High, because updates are to be notified to the user. |
| Frequency of Use: | Performed as much time as SubAdmin wants until the pothole is resolved completely. |
| Normal Course of Events: | 1. SubAdmin views the complaint.<br>2. Checks the status of pothole by contacting the contractor.<br>3. Updates the status of respective complaint on portal. |
| Alternative Courses: | F04.AC.1-SubAdmin only views the status but doesn't update its status. |
| Exceptions: | F04.EX.1-SubAdmin is unable to update the status. |
| Includes: | This function is executed only by SubAdmin. |
| Assumptions: | SubAdmin should assume that complaints registered are of his/her ward only. |
| Notes and Issues: | 1. If any complaint is irrelevant or address is wrong, SubAdmin will not work on it to solve the pothole. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F05 | | |
| Use Case Name: | Add Suggestion | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 16-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | User |
| Description: | User can suggest a positive/negative feedback regarding the work, system, etc by submitting a suggestion to the portal. |
| Preconditions: | 1. User must be logged in.<br>2. User must have completed all the previous functions like UC.F02, UC.F03. |
| Priority: | Medium |
| Frequency of Use: | Executed only once after performing previous tasks. One User can submit suggestion only one time. |
| Normal Course of Events: | 1. User logins.<br>2. According to work done, experience, etc user submits the suggestion.<br>3. Suggestion Submitted. |
| Alternative Courses: | F05.AC.1-User doesn't submit any suggestion & logouts. |
| Exceptions: | F05.EX.1-Suggestion is unable to get stored in Database. |
| Includes: | Executed once by each User. |
| Assumptions: | System assumes for suggestion by user for improvement of system. |
| Notes and Issues: | A suggestion or normal feedback for a website is important and must be executed by each user. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F06 | | |
| Use Case Name: | View Registered Complaints Nearby | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 16-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | User |
| Description: | User can view other complaints that are from the same area or from the same region. |
| Preconditions: | 1. User must be logged in. |
| Postconditions: | If there are not nearby registered complaint, next section will not be displayed. |
| Priority: | Low |
| Frequency of Use: | User can view nearby complaints as much number of times he/she wants. |
| Normal Course of Events: | 1. User logins.<br>2. Goes to "Complaints nearby" section.<br>3. Views all registered nearby complaint. |
| Exceptions: | F06.EX.1-Unable to fetch data from Database to display nearby complaints. |
| Includes: | Executed by User. |
| Assumptions: | A User viewing nearby complaints can assume to see similar problems & complaints. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F07 | | |
| Use Case Name: | View All Complaint | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 16-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | User, SubAdmin, MainAdmin |
| Description: | All 3 actors can view all registered complaints. |
| Preconditions: | User/SubAdmin/MainAdmin must be logged in. |
| Postconditions: | User & SubAdmin can only read a registered complaint.<br>MainAdmin can read as well as delete any of the complaint. |
| Priority: | Low |
| Frequency of Use: | It can be used by any of the 3 actors and any number of times. |
| Normal Course of Events: | 1. User/SubAdmin/MainAdmin logins.<br>2. Goes to view menu option.<br>3. Views all Complaint. |
| Exceptions: | F07.EX.1-Database does not respond to display all complaints. |
| Includes: | Executed by any of 3 actors. |
| Special Requirements: | All Users can view complaints of other area too. |
| Notes and Issues: | Viewing all complaints can show detail of complaints to all 3 actors. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F08 | | |
| Use Case Name: | Delete Complaint | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 16-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | MainAdmin |
| Description: | Will be executed by MainAdmin as & when he/she needs to delete any of the complaint. |
| Preconditions: | 1. MainAdmin must be logged in.<br>2. MainAdmin should know which complaint to keep & which complaint to delete. |
| Postconditions: | After deleting a complaint, MainAdmin should refresh Database to keep record of valid complaints. |
| Priority: | Low |
| Frequency of Use: | Can be used anytime & any number of times as & when MainAdmin requires. |
| Normal Course of Events: | 1. MainAdmin logins.<br>2. Views all complaint.<br>3. Checks & deletes unwanted/fake/duplicate/etc complaint. |
| Alternative Courses: | F08.AC.1-MainAdmin logins but doesn't deletes any of complaint & logouts. |
| Exceptions: | F08.EX.1-MainAdmin is unable to delete any complaint. |
| Includes: | Executed by MainAdmin. |
| Notes and Issues: | Part of UC.F07 for MainAdmin. Important function to be executed by MainAdmin so that there is not a collection of waste complaint in the database. MainAdmin should maintain a track of deleted complaints as well. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F09 | | |
| Use Case Name: | Graph of Status/Progress | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 16-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | MainAdmin |
| Description: | All 3 actors can view the 3 graphs for better understanding of pothole data. |
| Preconditions: | 1.MainAdmin/SubAdmin/User should be logged in. |
| Priority: | High |
| Frequency of Use: | All 3 actors can any time view the graph. |
| Normal Course of Events: | 1. MainAdmin/SubAdmin/User logins.<br>2. Goes to Graph page.<br>3. Views the Report/Rating/Status Graph. |
| Alternative Courses: | F09.AC.1-Actor views the graph but can't understand the status. |
| Exceptions: | F09.EX.1-Graph doesn't display correct plotting according to data. |
| Includes: | Executed by all 3 actors. |
| Special Requirements: | According to MainAdmin, he/she can display any type of graph/chart. |
| Assumptions: | All 3 actors can assume that the graphs displayed are of correct data. |
| Notes and Issues: | A very important function as public visits on portal & sees the progress of system. Also a progress graph is important for a system. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F10 | | |
| Use Case Name: | Assign/Dismiss SubAdmin | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 16-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | MainAdmin |
| Description: | MainAdmin can assign or dismiss SubAdmin according to area requirements or non-requirements respectively. |
| Preconditions: | 1. MainAdmin must be logged in.<br>2. MainAdmin should have a record of area and their respective SubAdmin. |
| Postconditions: | MainAdmin should know for which area SubAdmin should be assigned and for which area SubAdmin should be dismissed. |
| Priority: | Medium |
| Frequency of Use: | Executed by MainAdmin according to area requirements only when required. |
| Normal Course of Events: | 1. MainAdmin logins.<br>2. Assigns area to a SubAdmin OR dismisses a SubAdmin. |
| Exceptions: | F10.EX.1-MainAdmin is unable to add or delete SubAdmin. |
| Includes: | Executed by MainAdmin according to requirements. |
| Assumptions: | If there are no complaints registered in a particular area, MainAdmin can assume to dismiss that area's SubAdmin. |
| Notes and Issues: | Executed by MainAdmin only when required. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F11 | | |
| Use Case Name: | Delete User Account | | |
| Created By: | Vicky Gupta | Last Updated By: | Vicky Gupta |
| Date Created: | 16-09-2018 | Date Last Updated: | 13-03-2019 |

| | |
|---|---|
| Actor: | MainAdmin |
| Description: | MainAdmin can delete some accounts that are fake/not registered any complaint/duplicate account or another case. |
| Preconditions: | 1. MainAdmin must be logged in.<br>2. MainAdmin should have a track of all users account created. |
| Postconditions: | 1. MainAdmin should refresh database after deleting an account. |
| Priority: | Medium |
| Frequency of Use: | MainAdmin will decide how frequently this function should be used. |
| Normal Course of Events: | 1. MainAdmin logins.<br>2. Checks all user accounts.<br>3. Deletes user account if not needed. |
| Alternative Courses: | F11.AC.1-MainAdmin checks all user accounts but doesn't delete any account. |
| Exceptions: | F11.EX.1-MainAdmin is unable to delete any account. |
| Includes: | Executed by MainAdmin. |
| Assumptions: | MainAdmin can assume to have real & valid user accounts so that there is no need to delete any user account. |
| Notes and Issues: | By executing this function, MainAdmin can maintain well & good track of real/valid user accounts on his/her system. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F12 | | |
| Use Case Name: | Add/Remove Ward | | |
| Created By: | Vicky Gupta | Last Updated By: | |
| Date Created: | 13-03-2019 | Date Last Updated: | |

| | |
|---|---|
| Actor: | MainAdmin |
| Description: | MainAdmin can add or remove ward according to permission of services granted to website. |
| Preconditions: | 1. MainAdmin must be logged in.<br>2. MainAdmin should know which ward to be added and which to be removed. |
| Postconditions: | 1. MainAdmin should assign/dismiss SubAdmin to/of respective ward. |
| Priority: | Medium. |
| Frequency of Use: | MainAdmin will decide how frequently this function should be used. |
| Normal Course of Events: | 1. MainAdmin logins.<br>2. Adds or Removes ward. |
| Alternative Courses: | F12.AC.1-MainAdmin checks which ward is required and which not to provide services. |
| Exceptions: | F12.EX.1-MainAdmin is unable to add/remove ward. |
| Includes: | Executed by MainAdmin. |
| Assumptions: | MainAdmin can assume to have solved complaint so that he/she can add more wards for services. |
| Notes and Issues: | By executing this function, MainAdmin can maintain well & good status of the System. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F13 | | |
| Use Case Name: | Reports | | |
| Created By: | Vicky Gupta | Last Updated By: | |
| Date Created: | 13-03-2019 | Date Last Updated: | |

| | |
|---|---|
| Actor: | User/SubAdmin/MainAdmin |
| Description: | User/SubAdmin/MainAdmin can view the reports which display 2 categories of complaints, i.e. Solved and Unsolved. |
| Preconditions: | 1. Actor must be logged in. |
| Priority: | Medium |
| Frequency of Use: | Can be used any number of times and as many times. |
| Normal Course of Events: | 1. Actor logins.<br>2. Views reports of Solved and Unsolved complaints. |
| Alternative Courses: | F13.AC.1-Actor logins but no reports are displayed. |
| Exceptions: | F13.EX.1-Solved and Unsolved Complaints are displayed together. |
| Includes: | Executed by all 3 actors. |
| Assumptions: | All 3 actors can assume to have a well-defined format to display the reports. |
| Notes and Issues: | By executing this function, User/SubAdmin/MainAdmin will come to know that how many complaints are solved and how many complaints are unsolved. |

| | | | |
|---|---|---|---|
| Use Case ID: | UC.F14 | | |
| Use Case Name: | Search Complaint | | |
| Created By: | Vicky Gupta | Last Updated By: | |
| Date Created: | 13-03-2019 | Date Last Updated: | |

| | |
|---|---|
| Actor: | MainAdmin & SubAdmin |
| Description: | MainAdmin can search and delete any complaint if needed.<br>SubAdmin can search any complaint and update its status. |
| Preconditions: | 1. MainAdmin/SubAdmin must be logged in.<br>2. MainAdmin/SubAdmin should know what keyword and attribute is required to search a complaint. |
| Postconditions: | 1. MainAdmin should refresh database after searching if he/she deletes any complaint. |
| Priority: | Medium |
| Frequency of Use: | MainAdmin and SubAdmin should know how and when to use this function. |
| Normal Course of Events: | 1. MainAdmin/SubAdmin logins.<br>2. Searches a complaint using a keyword.<br>3. Deletes complaint/Updates status of the complaint. |
| Alternative Courses: | F14.AC.1-MainAdmin searches any complaint but doesn't delete it.<br>F14.AC.2-SubAdmin searches any complaint but doesn't update its status. |
| Exceptions: | F14.EX.1-MainAdmin/SubAdmin is unable to search a complaint. |
| Includes: | Executed by MainAdmin and SubAdmin. |
| Assumptions: | MainAdmin/SubAdmin can assume that there entered keyword according to attribute will display a list of complaints. |
| Notes and Issues: | By executing this function, MainAdmin/SubAdmin can quickly access to a complaint. |

## 4.2 Data Design

Data Design consists of how data is organized in the database, how it is managed and manipulated.

In our system's database, we will keep separate databases for all data required. Database structure is:-

Database Name – **pcs**

Tables ~

Name – **complaints**

Attributes – id, user_id, name, nop, cbname, street, locality, city, pincode, landmark, wardname, compstatus, created_at, updated_at, pothole_image.

Name – **users**

Attribute – id, name, user_id, email, password, created_at, updated_at.

Name – **wards**

Attribute – ward_id, ward_name

Name – **suggestions**

Attribute – id, user_id, name, content, stars.

Name – **sub_admins**

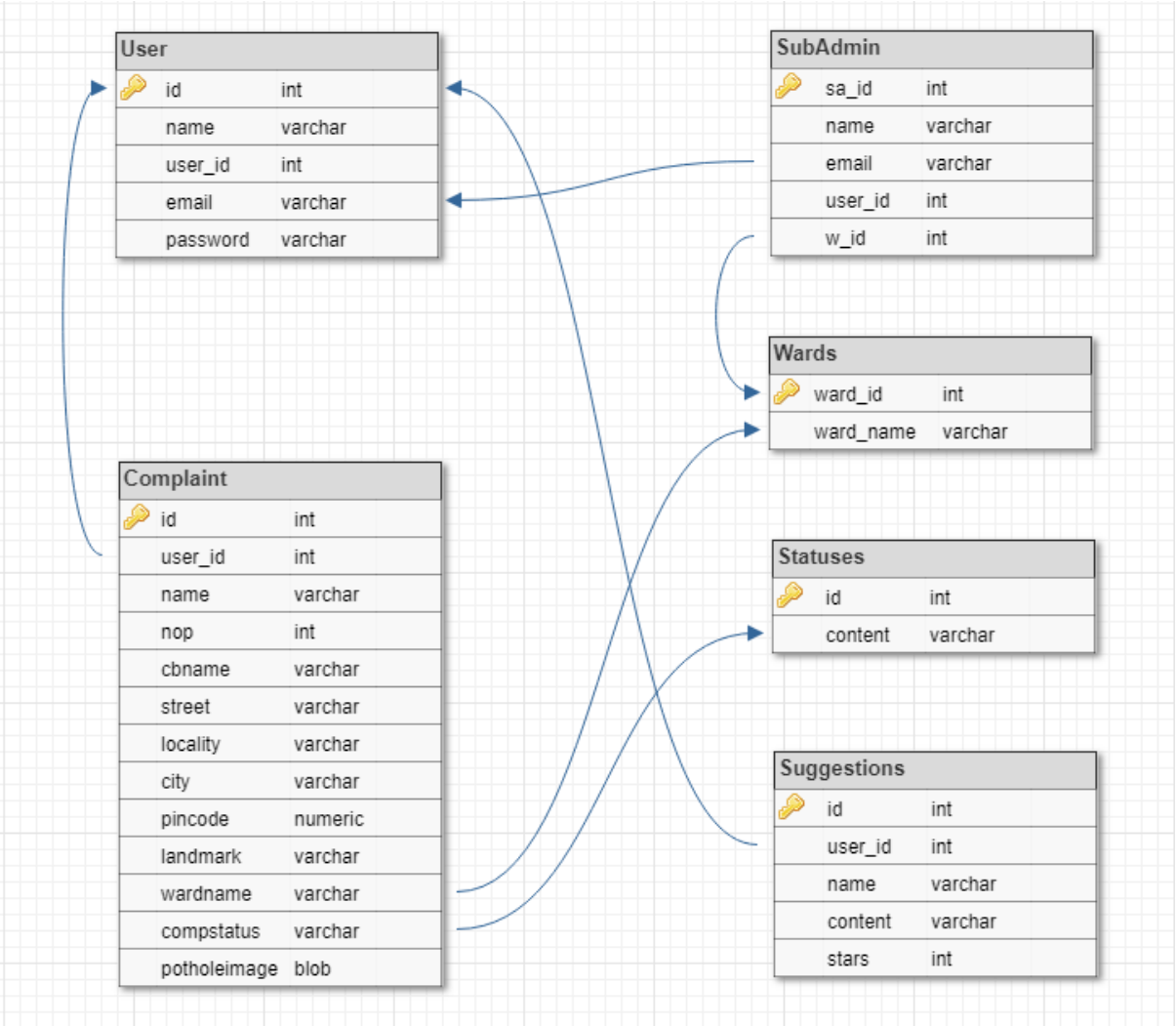
Attribute – sa_id, name, email, user_id, w_id,.

Name – **statuses**

Attribute – id, content

### 4.2.1 Schema Design

Schema used in this project is that the tasks of User, SubAdmin and MainAdmin are performed in a way that there all respective tasks interact with each other for better communication.



### 4.2.2 Data Integrity and Constraints

Data integrity is the maintenance and the assurance of the accuracy and consistency of, data over its entire life cycle, and is a critical aspect to the design, implementation and usage of any system which stores, processes, or retrieves data.

Attributes like complaint_id, ward_id, status_id, feedback_id, etc are kept as unique and not null. Some attributes can be kept null or not unique according to requirements.

## 4.3 Procedural Design

Procedural design is a systematic way for developing algorithms or procedurals. We can use procedural design to get an initial idea of the flow of a particular function.

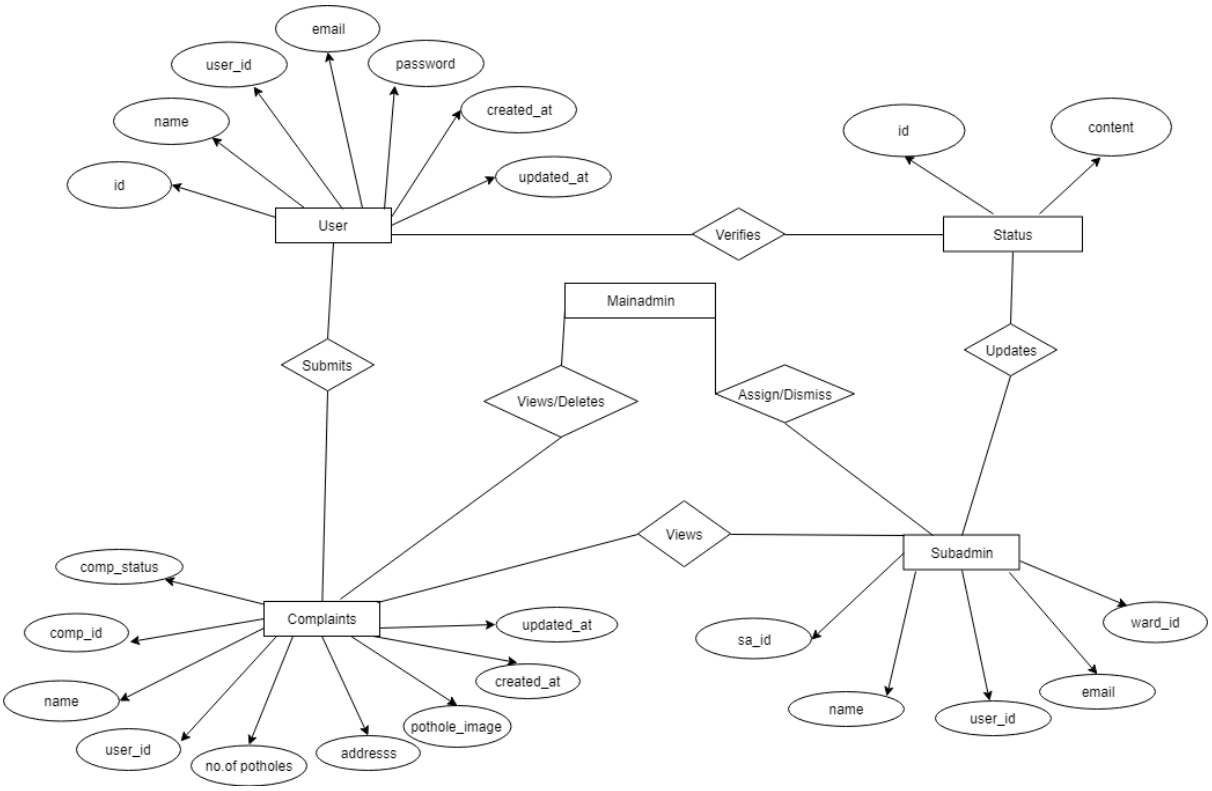
### 4.3.1 Logic Diagrams

We will be creating some UML(Unified Modeling Language) Diagrams to define the systematic flow of procedure that improves its comprehension and will help us during implementation. For example, Use Case Diagram, Entity Relationship(ER) Diagram, State Diagram.

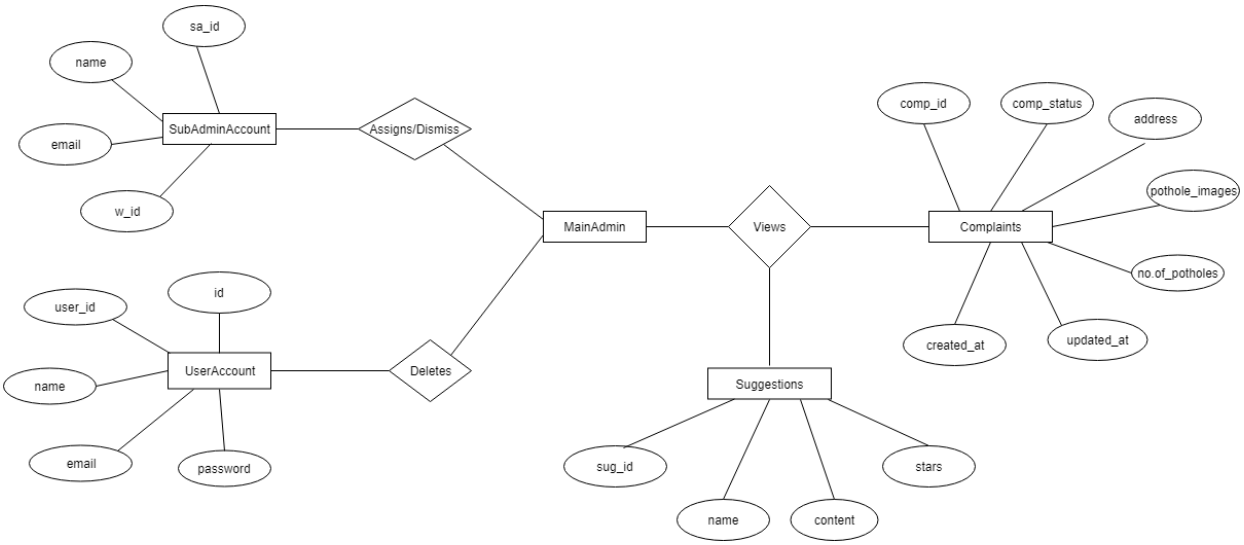
# UML Diagrams

## Entity Relationships Diagram

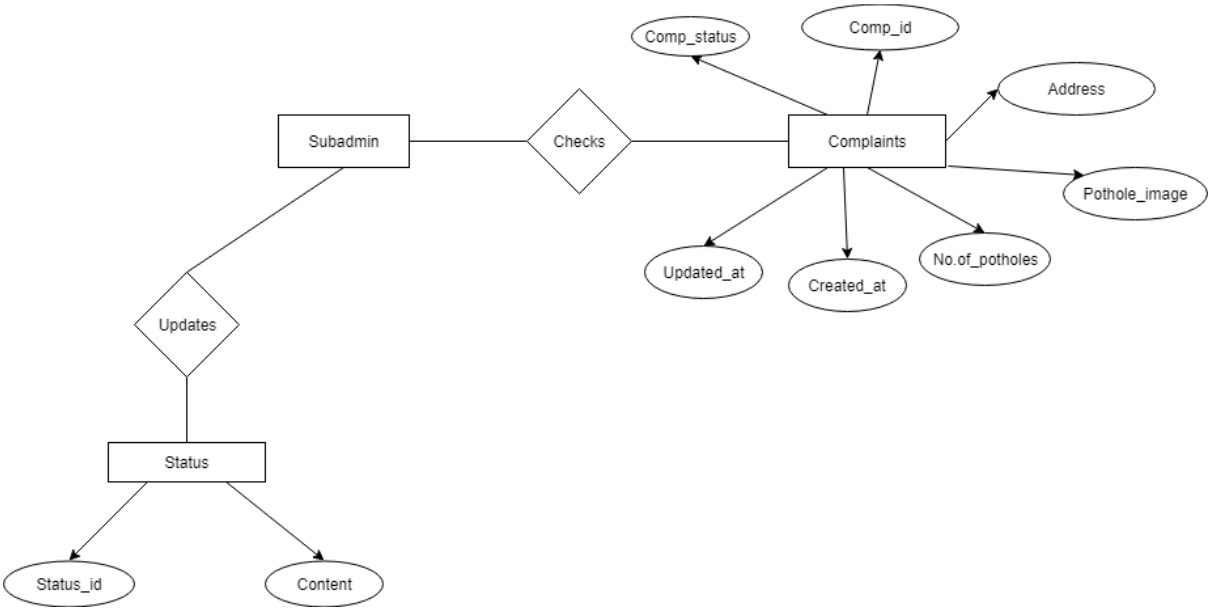
### Overall ER Diagram



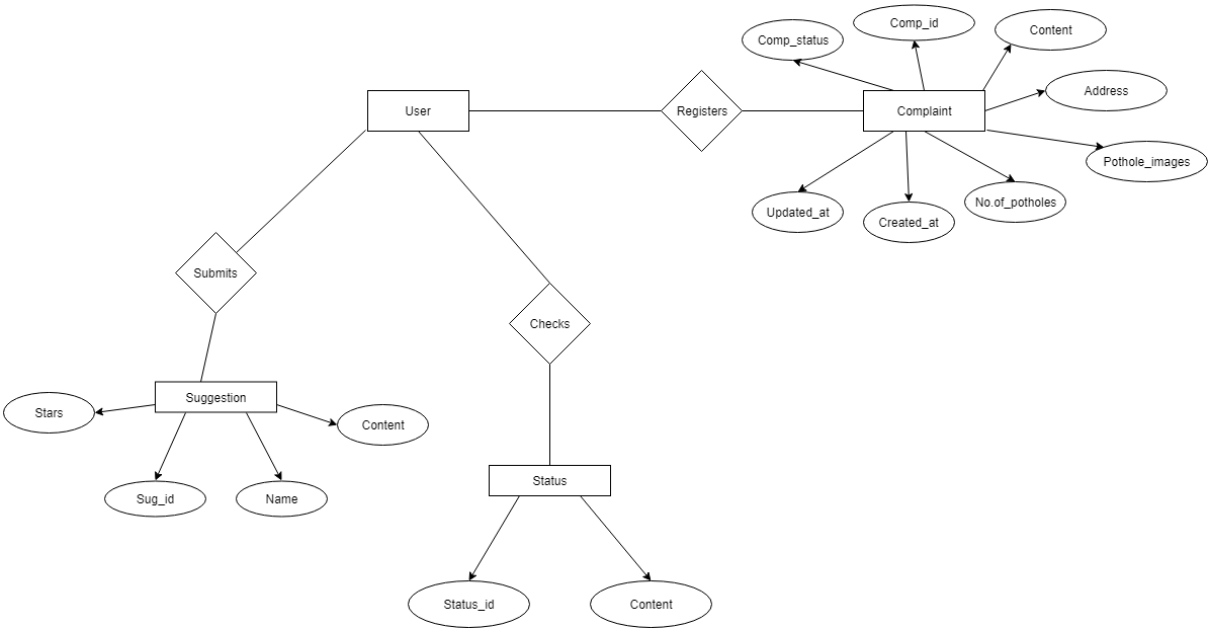

### MainAdmin ER Diagram



## SubAdmin ER Diagram

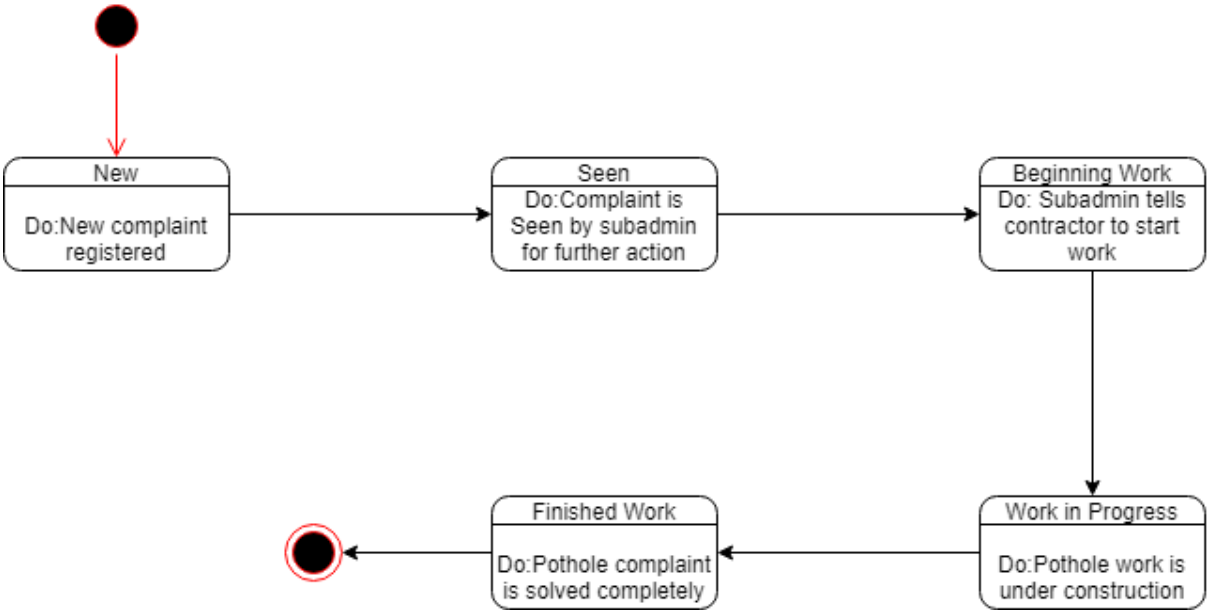



## User ER Diagram



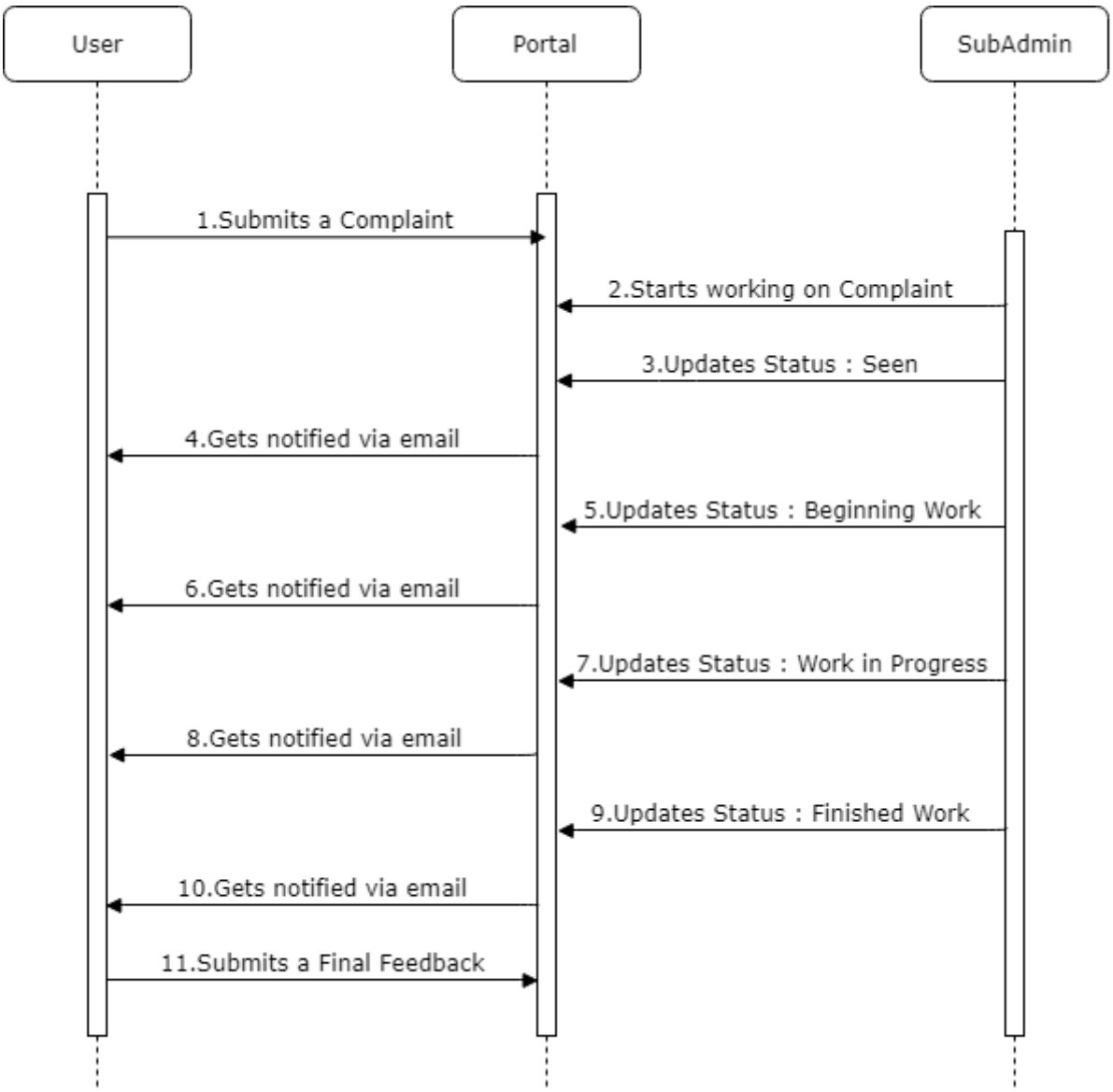
# State Diagram

## Complaint State Diagram



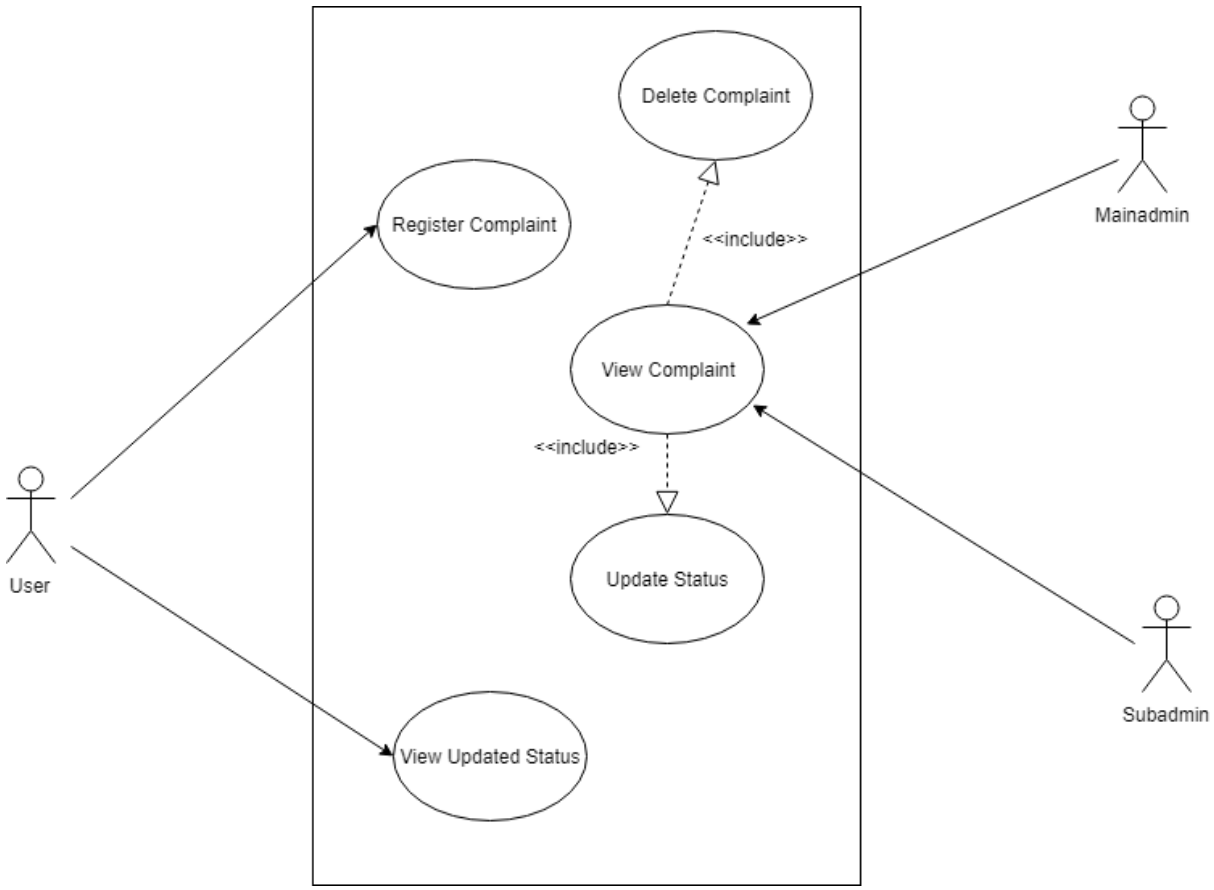
# Sequence Diagram

## Complaint Sequence Diagram



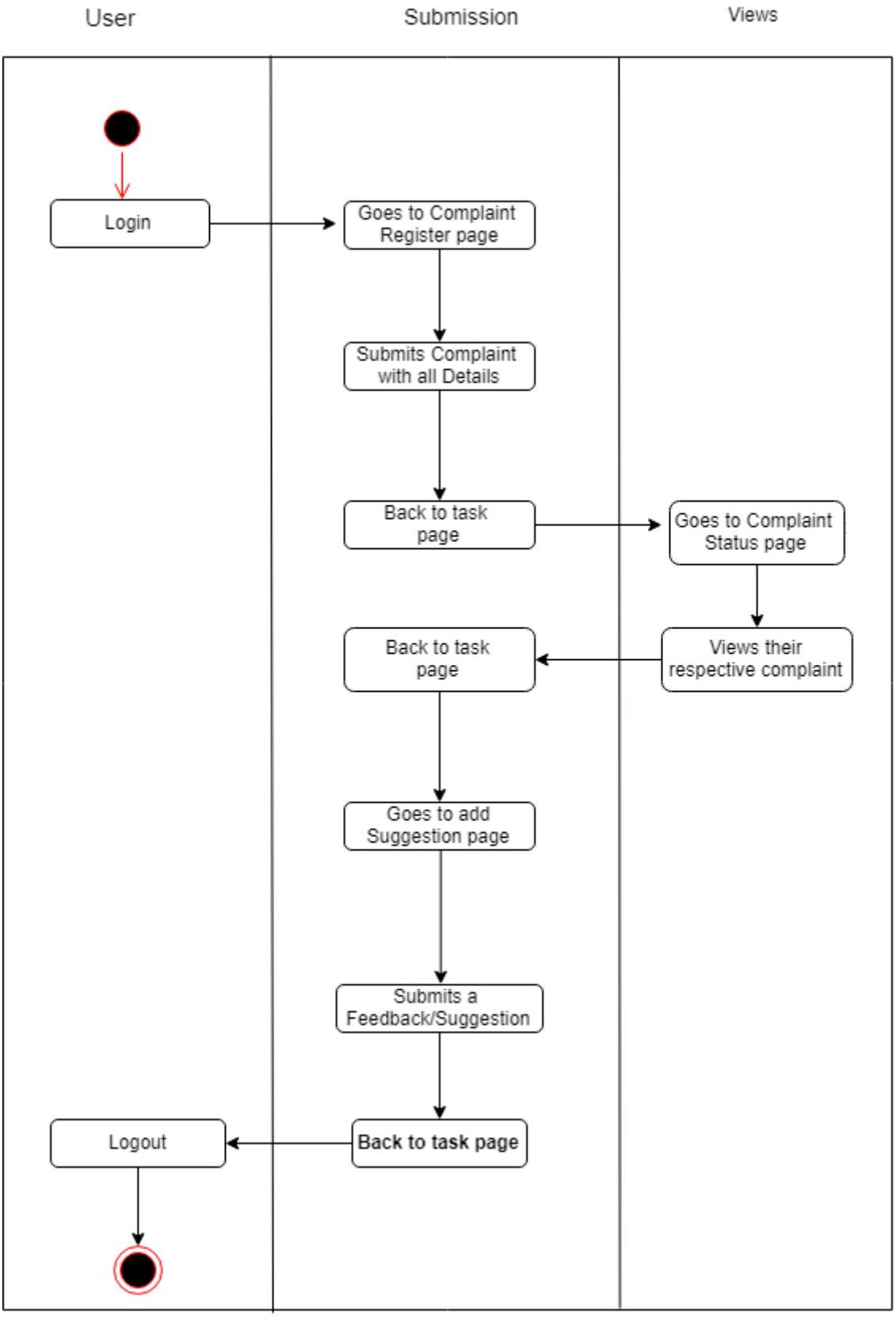
# Use Case Diagram

## Complaint Use Case Diagram

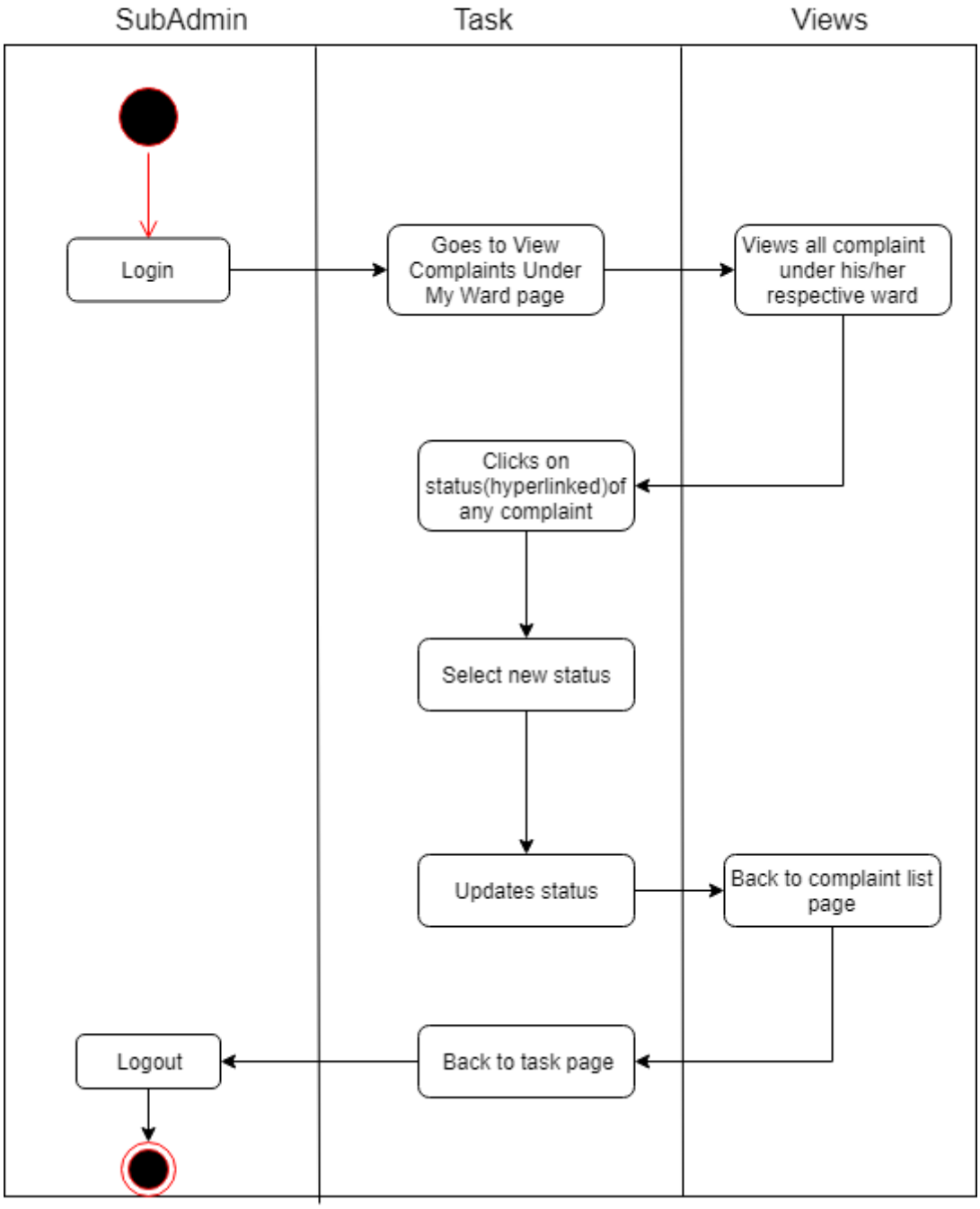


# Activity Diagram

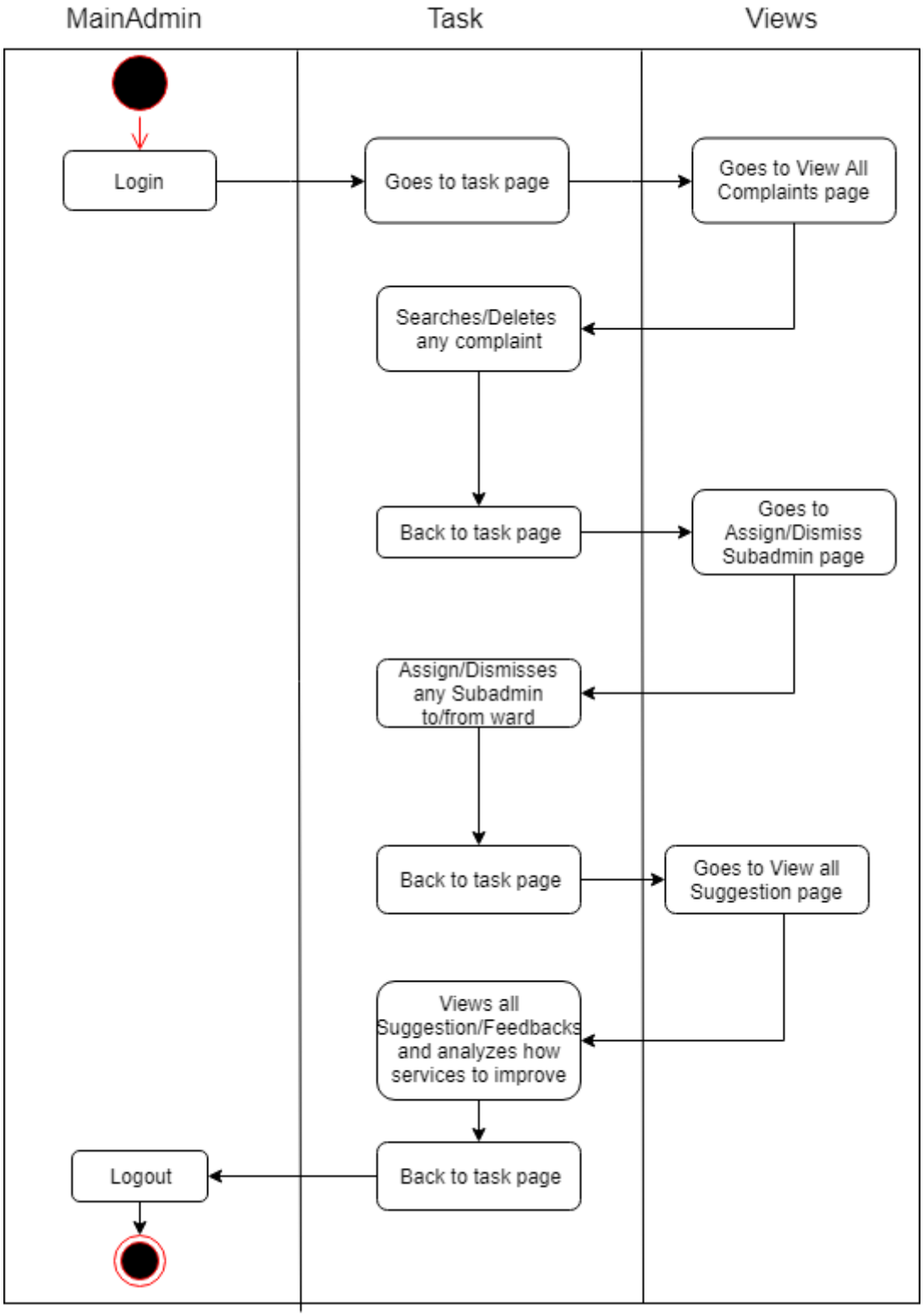
## User Activity Diagram



**SubAdmin Activity Diagram**

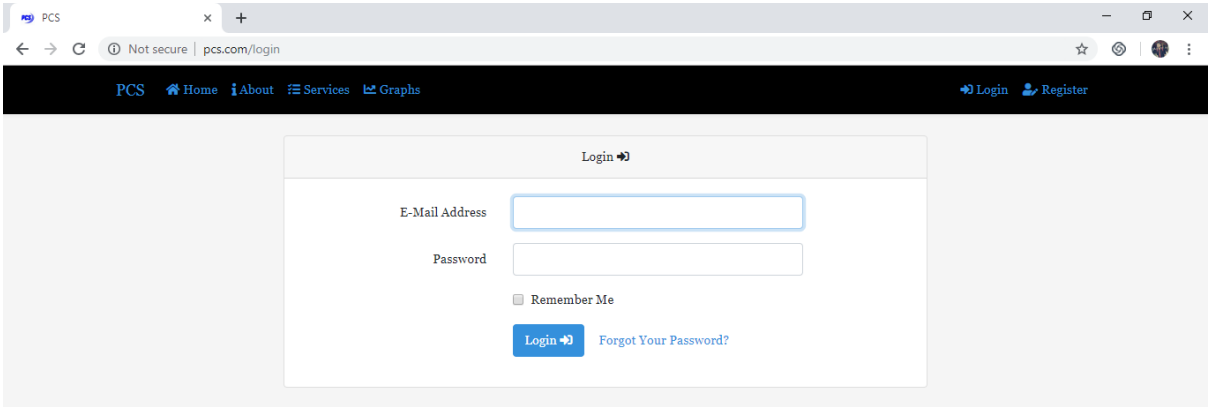


**MainAdmin Activity Diagram**

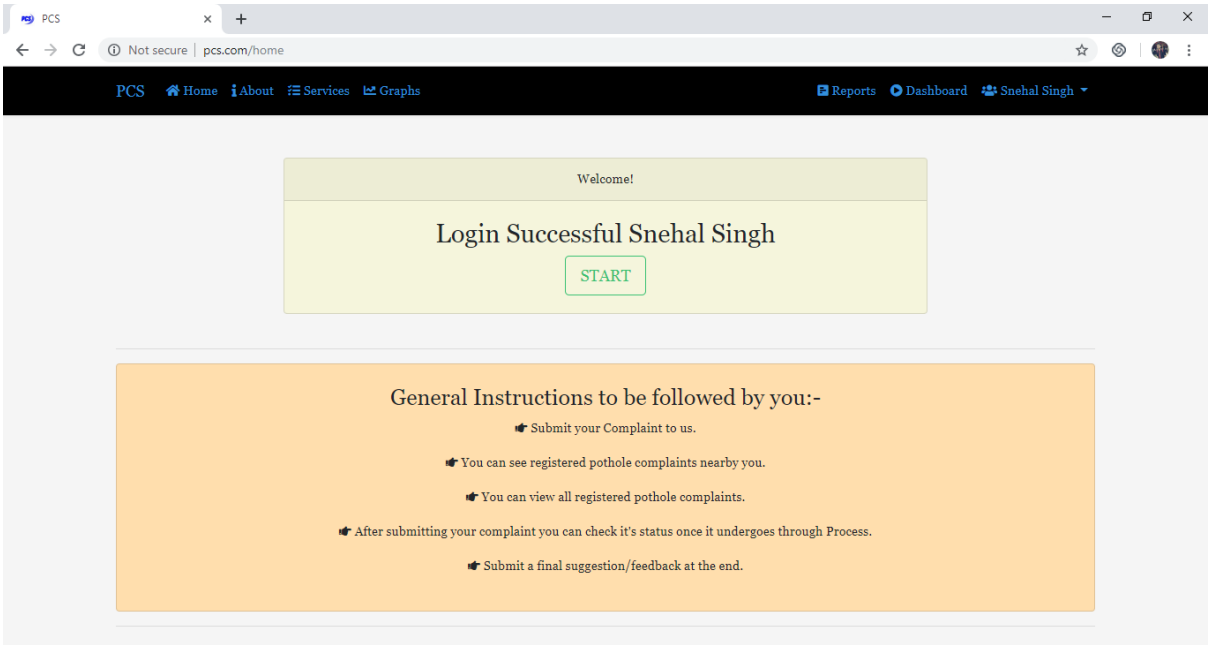


## 4.4 User interface design

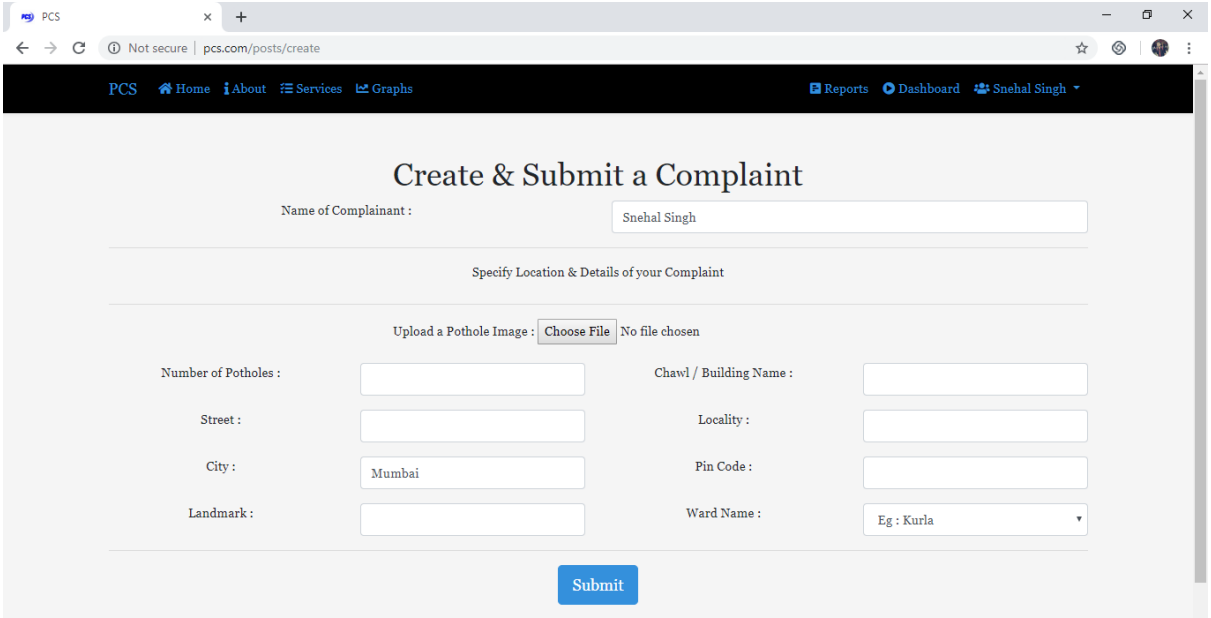
**Login Page**



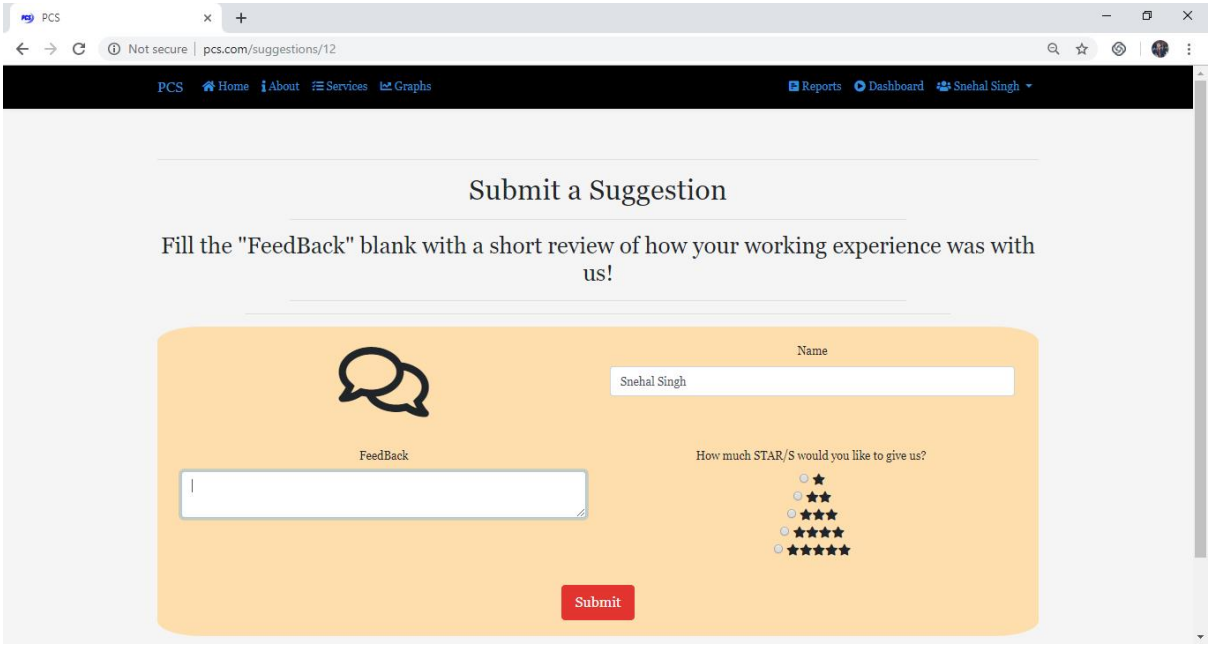

**Welcome Page**



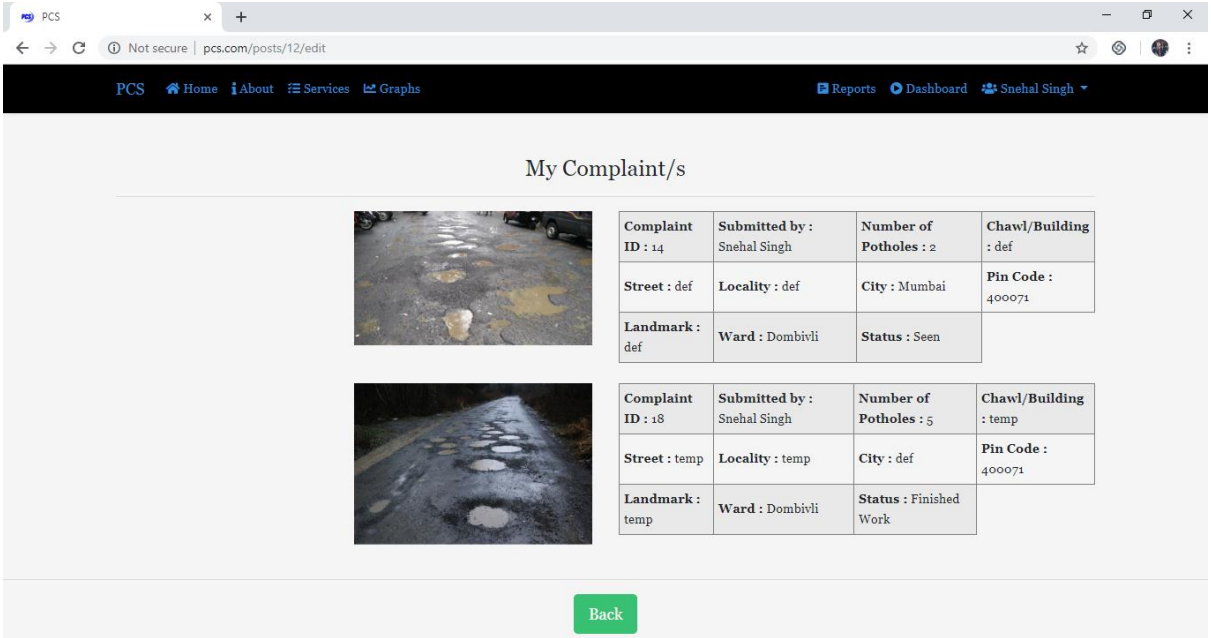
## Complaint Submit Page



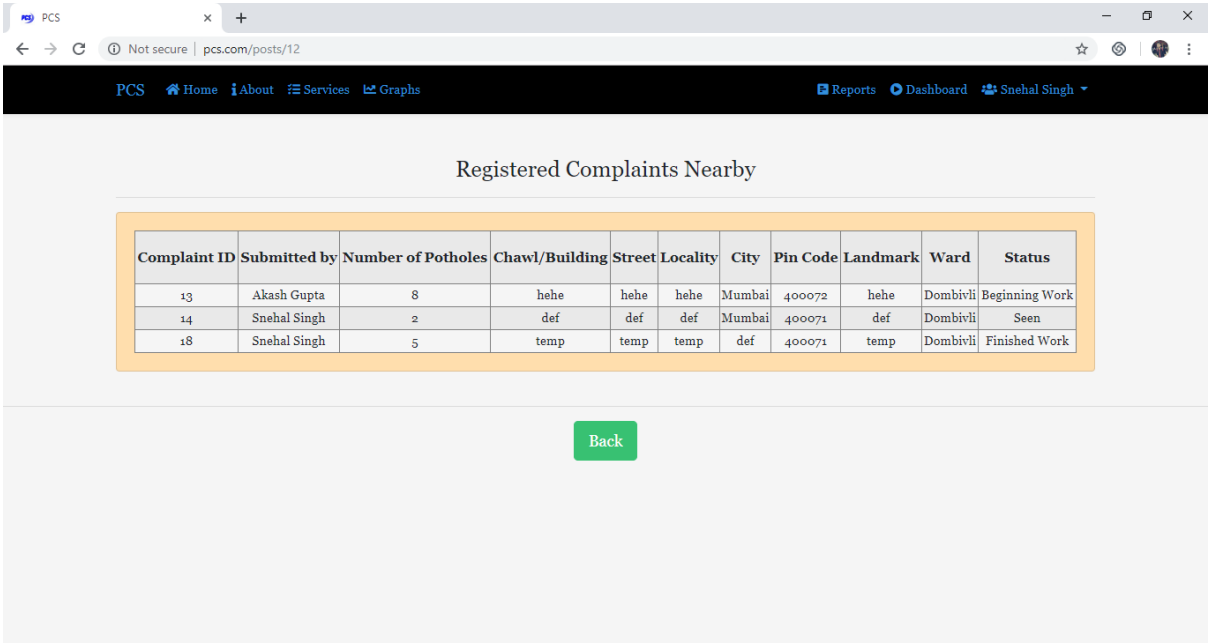

## Feedback Submit Page



## Viewing My Complaint/s Status




## Viewing Registered Complaints nearby User




| Complaint ID | Submitted by | Number of Potholes | Chawl/Building | Street | Locality | City | Pin Code | Landmark | Ward | Status |
|---|---|---|---|---|---|---|---|---|---|---|
| 13 | Akash Gupta | 8 | hehe | hehe | hehe | Mumbai | 400072 | hehe | Dombivli | Beginning Work |
| 14 | Snehal Singh | 2 | def | def | def | Mumbai | 400071 | def | Dombivli | Seen |
| 18 | Snehal Singh | 5 | temp | temp | temp | def | 400071 | temp | Dombivli | Finished Work |

## 4.5 Security Issues

As we are not using Java or any other secured programming language to code in our project, we can say that data is not much safe & secure. It assures the users of the website that their data is secured. Laravel is a standout amongst the most well known and advanced PHP frameworks for web applications with its expressive syntax.

Laravel provides various mechanisms to secure website. Some of are listed below:

- *Encryption*
- *Storing Password*
- *Authenticating Users*
- *Cross-site request forgery (XSS)*
- *Avoiding SQL injection*
- *Protecting Routes*
- *HTTP Basic Authentication*

## 4.6 Test Cases Design

**Test Case ID:** TC_001

**Test Case Name:** Registration/Sign Up

**Description:** To use our portal user should first register and create their account.

**Prerequisite:** The user must be new and have proper email ID.

| Step | Test Step | Test Data | Expected Result | Actual Result | Status | Notes |
|---|---|---|---|---|---|---|
| 1 | Goes to sign up page | | Sign Up page should be open. | Sign up page is open. | Pass | |
| 2 | Enter details | Username: Tarun<br>Password:verma123<br>Email ID:tarun@gmail.com | Details entered successfully | Details entered successfully. | Pass | |
| 3 | Click on register button | | Registration done successfully | Registration done successfully | Pass | Prompt message should be given to user. |

**Test Case ID:** TC_002

**Test Case Name:** Login

**Description:** To enter into our portal user must have proper credential.

**Prerequisite:** The user must be registered.

| Step | Test Step | Test Data | Expected Result | Actual Result | Status | Notes |
|---|---|---|---|---|---|---|
| 1 | Goes to Login page | | Login page should open. | Login page is open. | Pass | |
| 2 | Enter the details | Email ID: tarun@gmail.com<br><br>Password:verma123 | Details entered successfully. | Details entered successfully. | Pass | |
| 3 | Click on login button | | Login successful. | Login successful. | Pass | User gets redirected to welcome page |

**Test Case ID:** TC_003

**Test Case Name:** Register Complaint

**Description:** User will submit a complaint report of a pothole.

**Prerequisite:** User must be logged in.

| Step | Test Step | Test Data | Expected Result | Actual Result | Status | Notes |
|---|---|---|---|---|---|---|
| 1 | Comes on registering complaint page | | Page should open. | Page is open. | Pass | |
| 2 | Enter the details | Photo, address, Landmark, no of potholes, pincode, etc | Complaint entered successfully. | Complaint entered successfully. | Pass | |
| 3 | Click on submit button | | Complaint registered successfully. | Complaint registered successfully. | Pass | Prompt message to display Complaint Submitted Successfully. |

**Test Case ID:** TC_004

**Test Case Name**: View All Complaint

**Description:** User/SubAdmin/MainAdmin can view all registered complaints.

**Prerequisite:** Actor must be logged in.

| Step | Test Step | Test Data | Expected Result | Actual Result | Status | Notes |
|---|---|---|---|---|---|---|
| 1 | Goes to "View All Complaints" page. | | Page should open. | Page is open. | Pass | |
| 2 | Views all the complaint. | | Page displays all registered complaints. | Page displays all complaints. | Pass | Page will display complaints only if there are previously registered complaints. |

**Test Case ID:** TC_005

**Test Case Name:** Graph of Status/Progress

**Description:** User/SubAdmin/MainAdmin can view graph which displays rating, reports and status of all complaints.

**Prerequisite:** MainAdmin/SubAdmin/User should be logged in.

| Step | Test Step | Test Data | Expected Result | Actual Result | Status | Notes |
|---|---|---|---|---|---|---|
| 1 | Clicks on Graph button present in navbar. | | Graph page opens up. | All 3 graph displays correct plotting of data. | Pass | |

**Test Case ID**: TC_006

**Test Case Name:** Add Suggestion/Submit Feedback

**Description:** User after interacting with SubAdmin on portal from 1st to last step will submit a feedback type suggestion(positive or negative).

**Prerequisite:** User must be logged in.

| Step | Test Step | Test Data | Expected Result | Actual Result | Status | Notes |
|---|---|---|---|---|---|---|
| 1 | Clicks on "Add Suggestion" tab. | | Add Suggestion tab should open. | Add Suggestion tab opens. | Pass. | Prompts to enter suggestion(optional). |
| 2 | Enters suggestion. | Statement, stars. | Suggestion entered successfully. | Suggestion entered successfully. | Pass. | |
| 3 | Click on the Send button. | | Suggestion submitted successfully. | Suggestion submitted successfully. | Pass. | Prompts that User can now Logout. |

**Test Case ID**: TC_007

**Test Case Name:** Update Status

**Description:** SubAdmin will view complaints under his/her ward and update status of complaints according to their progress.

**Prerequisite:** SubAdmin should be logged in. Complaints must be registered under his/her ward.

| Step | Test Step | Test Data | Expected Result | Actual Result | Status | Notes |
|---|---|---|---|---|---|---|
| 1 | SubAdmin views all complaints under him/her. | | All complaints get listed. | All complaints get displayed in tabular format. | Pass. | |
| 2 | Clicks on any status link of a complaint. | | A detail of Status gets displayed on new page. | Name, wardname and status(dropdown ) is displayed. | Pass. | |
| 3 | Clicks on dropdown button and selects new status. | | New status gets selected. | New status gets selected. | Pass. | |
| 4 | Clicks on Update button. | | Status gets updated. | Status gets updated in tabular display too. | Pass | SubAdmin should go back to list page to see updated status. |

# Chapter 5

# Implementation and Testing

## 5.1 Implementation Approaches

For making this project complete working model, we have divided all the functionalities into small and various modules. Modules which are made in sequence and by observing their dependencies on other modules are described in Gantt chart in 3rd Chapter Requirement and Analysis. Since many of modules are common for all actors and hence they are made after all different modules.

Standard used for implementation of modules are Top down approach. In top down approach-

1. The whole problem/functionality is splitted into two or more modules.

2. Start implementing those modules.

3. If these modules are not easy to be implemented as a whole, split them further and implement them with a better solution.

4. Merge the implemented modules for a functionality to be executed.

## 5.2 Coding Details and Code Efficiency

### 5.2.1 Code Efficiency

In this section, not all but some codes and program will be explained using comments, screenshots of inputs and outputs. It will describe how code is efficient and how does it works.

Module – **Search Complaint**.

Performed by - MainAdmin

*Search/index.blade.php*

```
<!-- Search and Delete Complaint Code-->
   <div align="center" class="container">
      <h2>Search for a Complaint</h2>
      <hr style="background-color:red;width:80%">
      <p style="font-size:2.5em">Search a <b>Complaint</b> using <b>Keyword</b></p>
      {!! Form::open(['action' => 'SearchController@store','method'=>'POST']) !!}
         <table cellpadding=5>
            <tr>
               <!-- Dropdown input for key -->
               <td>{{Form::label('opt','Search by :')}}</td>
               <td><select class="form-control" name="opt">
                     <option value="street">Street</option>
                     <option value="locality">Locality</option>
                     <option value="landmark">Landmark</option>
                     <option value="wardname">Wardname</option>
                     <option value="pincode">Pin Code</option>
                  </select>
               </td>
               <!-- Text input for keyword -->
               <td>{{Form::label('input','Enter keyword here :')}}</td>
               <td>{{Form::text('input','',['class'=>'form-control'])}}</td>
            </tr>
         </table>
         <!-- Search button to perform operation -->
         {{Form::submit('Search',['class'=>'btn btn-info'])}}
      {!! Form::close() !!}
      <hr style="background-color:red;width:80%">
   </div>
```

*SearchController.php*

```
public function store(Request $request)
   {
      //validates both the inputs via dropdown and textinput
      $this->validate($request,[
         'opt'=>'required',
```

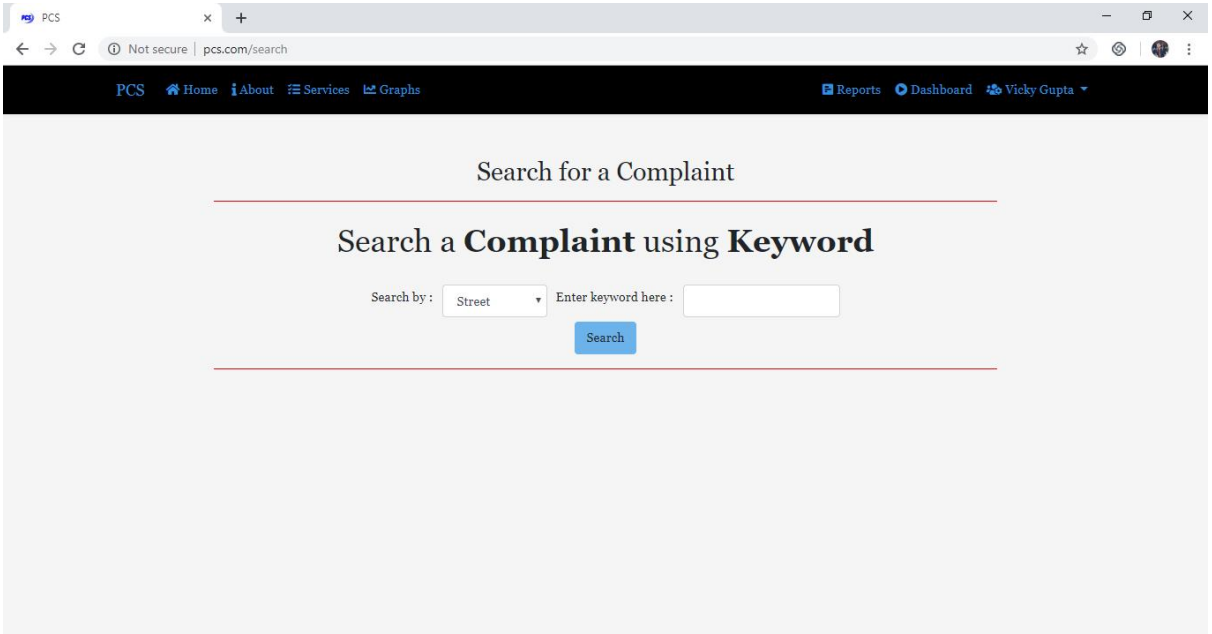
```
        'input'=>'required'
    ]);
    //stores values of both inputs in different object
    $ob1 = $request->input('opt');
    $ob2 = $request->input('input');
    //condition is applied on complaint table using
    //both inputs and result is stored in new object
    $output = DB::table('complaints')
    ->select('complaints.*')
    ->where($ob1, 'LIKE', "%{$ob2}%")
    ->get();
    //retrieved data is sent to the webpage by passing the object
    return view('search.show',['ob2'=>$ob2])->with('output',$output);
  }
```

*Search/show.blade.php*

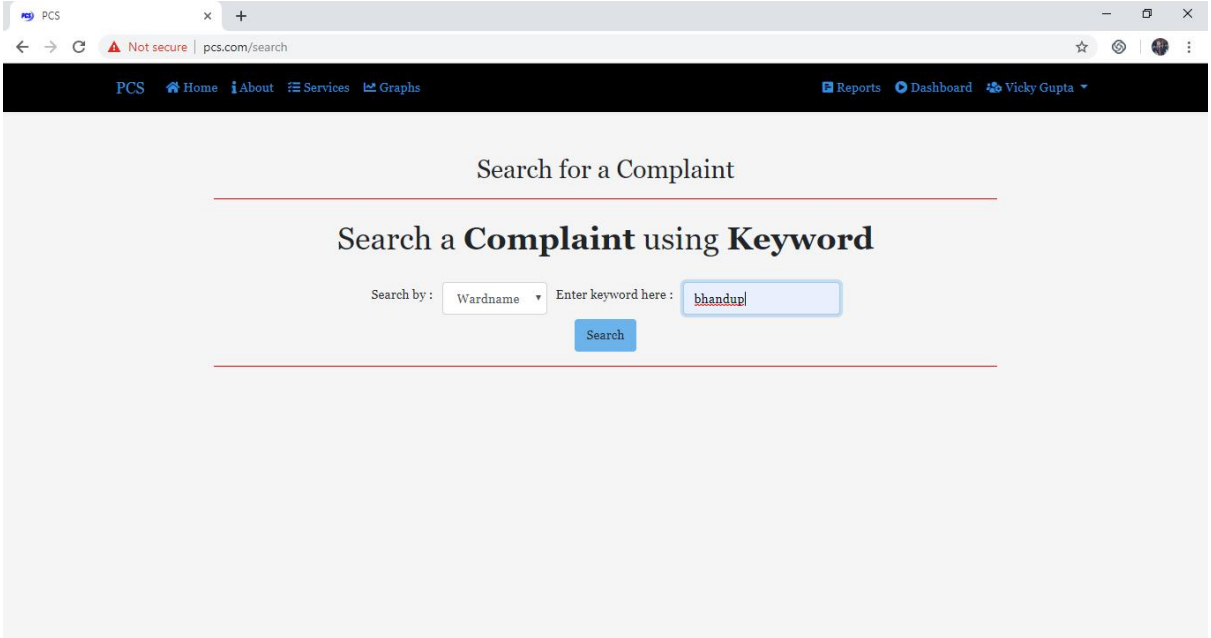
```
<!-- Code for Output results of Search Complaint -->
  <div align="center" class="container">
    <h2>Search Results</h2>
    <hr style="background-color:crimson;width:50%">
    <h3>Filtering Records by "{{$ob2}}"</h3>
    <hr style="background-color:crimson;width:50%">
    <br>
    <!-- Checking if output has records 1 or more than 1 -->
    @if( count($output) > 0)
    <div class="card" style="padding:20px;background-color:navajowhite">
      <!-- Table view used to display filtered records -->
      <table style="background-color:whitesmoke" cellpadding=5 frame="box" rules="all"
class="table-striped table-hover">
        <!-- Header part of table -->
        <tr align="center" style="height:50px;font-size:16px">
          <td><b>Complaint ID</b></td>
          <td><b>Submitted by</b></td>
          <td><b>Number of Potholes</b></td>
          <td><b>Chawl/Building</b></td>
          <td><b>Street</b></td>
          <td><b>Locality</b></td>
          <td><b>City</b></td>
          <td><b>Pin Code</b></td>
          <td><b>Landmark</b></td>
          <td><b>Ward</b></td>
          <td><b>Status</b></td>
          <td></td>
        </tr>
        <!-- Records to be displayed -->
        @foreach($output as $cmp)
        <tr align="center">
          <td>{{$cmp->id}}</td>
          <td>{{$cmp->name}}</td>
          <td>{{$cmp->nop}}</td>
          <td>{{$cmp->cbname}}</td>
```

```
                <td>{{$cmp->street}}</td>
                <td>{{$cmp->locality}}</td>
                <td>{{$cmp->city}}</td>
                <td>{{$cmp->pincode}}</td>
                <td>{{$cmp->landmark}}</td>
                <td>{{$cmp->wardname}}</td>
                <td>{{$cmp->compstatus}}</td>
                @if(Auth::user()->user_id == 2)
                <td align="center">
                   {!!Form::open(['action'=>['PostsController@destroy',$cmp-
>id],'method'=>'POST'])!!}
                      {{Form::hidden('_method','DELETE')}}
                      {{Form::submit('Delete',['class'=>'btn btn-danger'])}}
                   {!!Form::close()!!}
                </td>
                @endif
             </tr>
             @endforeach
             <hr style="background-color:chartreuse;width:80%">
          </table>
       </div>
       <!-- If output has no records -->
       @else
          <hr style="background-color:chartreuse;width:80%">
          <h3>No Complaints Found by keyword "{{$ob2}}"</h3>
       @endif
       <hr style="background-color:chartreuse;width:80%">
    </div>
```

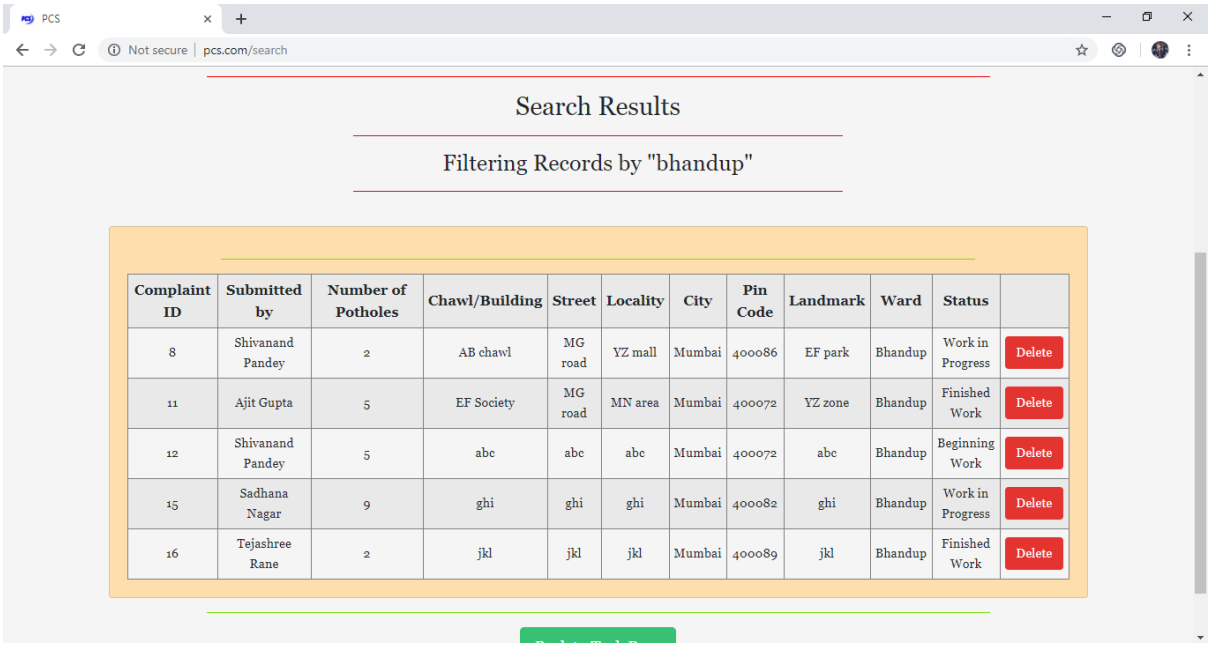
Input Page-



Selecting value from dropdown and entering value for input-




Output after clicking on Search button-




| Complaint ID | Submitted by | Number of Potholes | Chawl/Building | Street | Locality | City | Pin Code | Landmark | Ward | Status | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 8 | Shivanand Pandey | 2 | AB chawl | MG road | YZ mall | Mumbai | 400086 | EF park | Bhandup | Work in Progress | Delete |
| 11 | Ajit Gupta | 5 | EF Society | MG road | MN area | Mumbai | 400072 | YZ zone | Bhandup | Finished Work | Delete |
| 12 | Shivanand Pandey | 5 | abc | abc | abc | Mumbai | 400072 | abc | Bhandup | Beginning Work | Delete |
| 15 | Sadhana Nagar | 9 | ghi | ghi | ghi | Mumbai | 400082 | ghi | Bhandup | Work in Progress | Delete |
| 16 | Tejashree Rane | 2 | jkl | jkl | jkl | Mumbai | 400089 | jkl | Bhandup | Finished Work | Delete |

Module – **Update Status of a Complaint**.

Performed by – SubAdmin

*Saviews/edit.blade.php*

```
<!-- Code to update status of complaint -->
   <div class="container">
      <h2 align="center">Assign Status</h2>
      <!-- Using form control to display and retrieve values -->
      {!! Form::open(['action' => ['SataskController@update',$comp->id],'method'=>'POST'])
!!}
      <div class="form-group">
         {{Form::label('name','Name')}}
         {{Form::text('name',$comp->name,['class'=>'form-control'])}}
      </div>
      <div class="form-group">
         {{Form::label('nop','Number of Potholes')}}
         {{Form::text('nop',$comp->nop,['class'=>'form-control'])}}
      </div>
      <div class="form-group">
         {{Form::label('wardname','Ward Name')}}
         {{Form::text('wardname',$comp->wardname,['class'=>'form-control'])}}
      </div>
      <div class="form-group">
         {{Form::label('stat','Status')}}
         <select class="form-control" name="stat">
            <!-- Using for loop to display all status options -->
            @foreach($st as $s)
               @if($s->id != 1)
                  <option value="{{$s->content}}">{{$s->content}}</option>
               @else
                  <option selected hidden value="{{$comp->comp_status}}">{{$comp-
>comp_status}}</option>
               @endif
            @endforeach
         </select>
      </div>
      <!-- Button to update the status -->
      <div align="center">
         {{Form::hidden('_method','PUT')}}
         {{Form::submit('Update',['class'=>'btn btn-lg btn-warning'])}}
      </div>
   </div>
```

*SataskController.php*

```
public function update(Request $request, $id)
   {
      //validating status input via dropdown
      $this->validate($request,[
         //'name'=>'required',
         //'nop'=>'required',
```

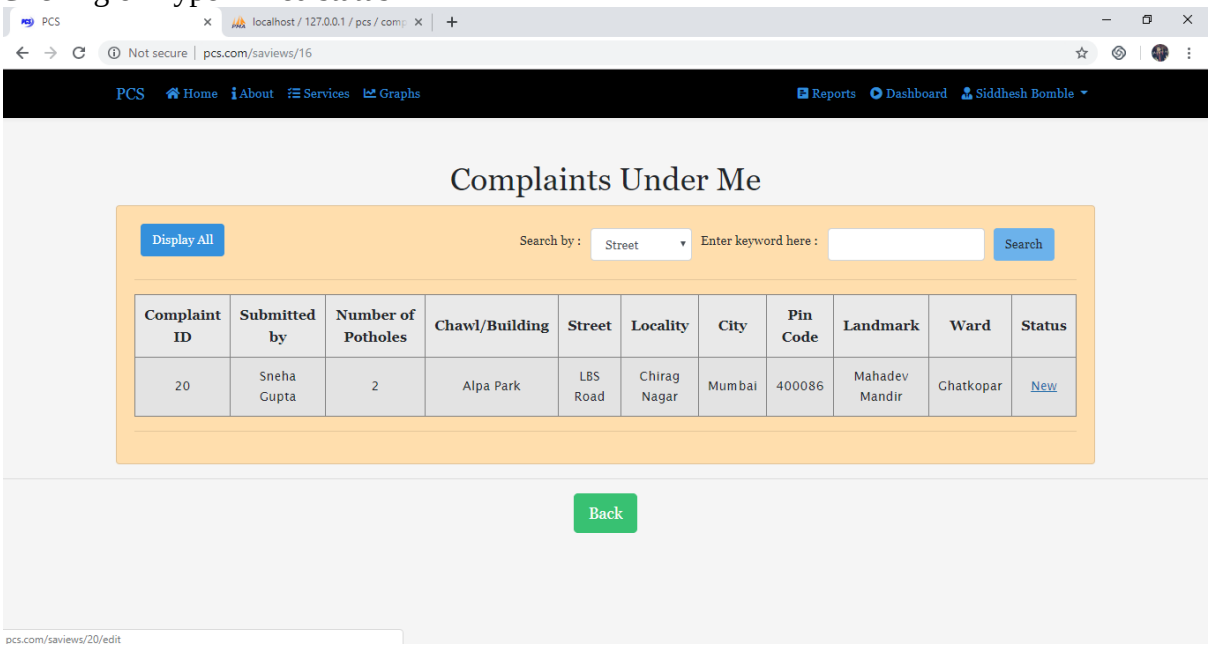
```
            //'wardname'=>'required',
            'stat'=>'required'
        ]);
        //searching id of the compaint and stroing it in object
         $comp = Complaint::find($id);
         //updating status of complaint using object of id and saving the changes
         $comp->compstatus = $request->input('stat');
         $comp->save();
         //retrieving all complaint and status in objects
         //and passing to webpage
         $comp = Complaint::find($id);
         $st = DB::table('statuses')
         ->select('statuses.*')
         ->get();
         return view('saviews.edit',['st'=>$st])->with('comp',$comp);
    }
```

*Saviews/show.blade.php*

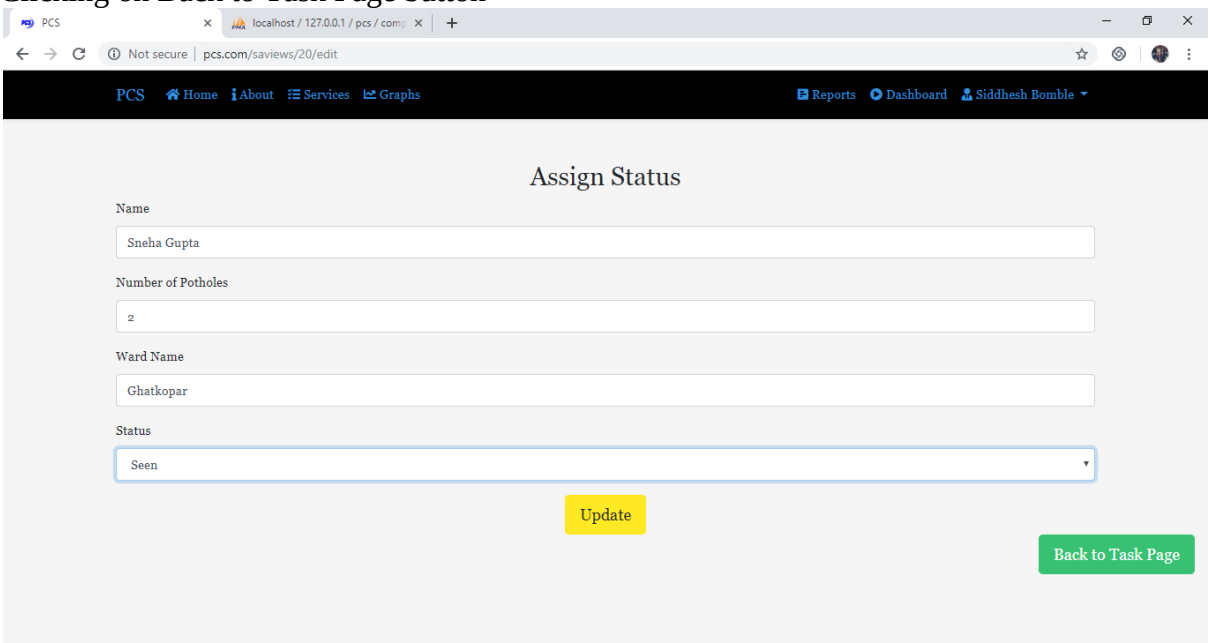
```
<!-- Using table to display complaints -->
            <table style="background-color:whitesmoke" frame="box" rules="all"
cellpadding=10 class="table-striped table-hover">
                <!-- Table Header -->
                <tr align="center" style="height:50px;font-size:16px">
                    <td><b>Complaint ID</b></td>
                    <td><b>Submitted by</b></td>
                    <td><b>Number of Potholes</b></td>
                    <td><b>Chawl/Building</b></td>
                    <td><b>Street</b></td>
                    <td><b>Locality</b></td>
                    <td><b>City</b></td>
                    <td><b>Pin Code</b></td>
                    <td><b>Landmark</b></td>
                    <td><b>Ward</b></td>
                    <td><b>Status</b></td>
                </tr>
                <!-- for loop to display complaints store in object -->
                @foreach($sql1 as $cmp)
                <tr align="center" style="height:50px;font-family:Lucida Sans">
                    <td>{{$cmp->id}}</td>
                    <td>{{$cmp->name}}</td>
                    <td>{{$cmp->nop}}</td>
                    <td>{{$cmp->cbname}}</td>
                    <td>{{$cmp->street}}</td>
                    <td>{{$cmp->locality}}</td>
                    <td>{{$cmp->city}}</td>
                    <td>{{$cmp->pincode}}</td>
                    <td>{{$cmp->landmark}}</td>
                    <td>{{$cmp->wardname}}</td>
                    <!-- Adding hyperlink to go on next page and update the status -->
                    <td><a href="/saviews/{{$cmp->id}}/edit">{{$cmp->compstatus}}</a></td>
                </tr>
```

```
    @endforeach
</table>
```

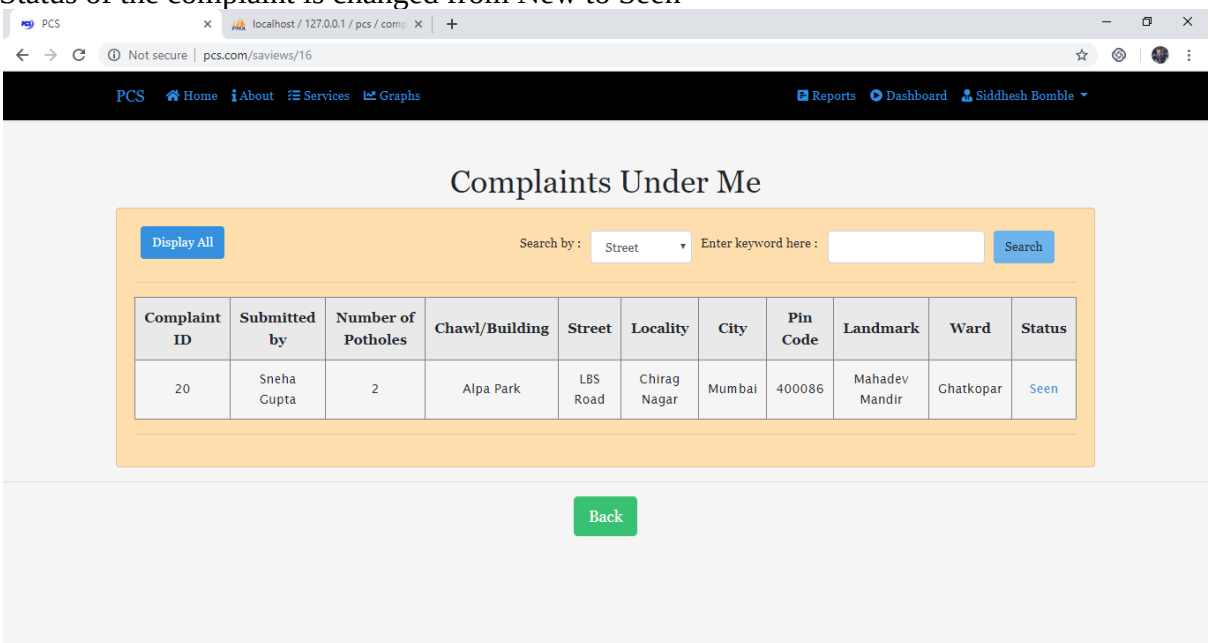
Clicking on hyperlinked status-




Selecting new status from dropdown and clicking on Update button-
Clicking on Back to Task Page button-



Status of the complaint is changed from New to Seen-



Module – **Submit Feedback/Add Suggestion**.

Performed by – User

*Suggestions/create.blade.php*

```
<div class="form" style="padding-right:30px;padding-left:30px;padding-
bottom:20px;padding-top:20px;border-radius:5%;background-color:navajowhite;">
    <!-- Form control to input feedback and stars -->
        {!! Form::open(['action' => 'SuggestionController@store','method'=>'POST']) !!}
        <div class="form-group row">
            <div class="form-group col-md-6">
                <i class="far fa-comments fa-7x"></i>
            </div>
            <div class="form-group col-md-6">
                {{Form::label('name','Name')}}
                {{Form::text('name',Auth::user()->name,['class'=>'form-control'])}}
            </div>
        </div>
        <div class="form-group row">
            <!-- Text input for feedback -->
            <div class="form-group col-md-6">
                {{Form::label('content','FeedBack')}}
                {{Form::textarea('content','',['class'=>'form-control','rows'=>2,'required
autofocus'])}}
            </div>
            <!-- Rating input using radio button -->
            <div class="form-group col-md-6">
            {{Form::label('stars','How much STAR/S would you like to give us?')}}
            <br>
            <input type="radio" name="stars" value="1"> <i class="fas fa-star"></i><br>
            <input type="radio" name="stars" value="2"> <i class="fas fa-star"></i><i
class="fas fa-star"></i><br>
            <input type="radio" name="stars" value="3"> <i class="fas fa-star"></i><i
class="fas fa-star"></i><i class="fas fa-star"></i><br>
            <input type="radio" name="stars" value="4"> <i class="fas fa-star"></i><i
class="fas fa-star"></i><i class="fas fa-star"></i><i class="fas fa-star"></i></i><br>
            <input type="radio" name="stars" value="5" required> <i class="fas fa-
star"></i><i class="fas fa-star"></i><i class="fas fa-star"></i><i class="fas fa-star"></i><i
class="fas fa-star"></i><br>
            </div>
        </div>
        {{Form::submit('Submit',['class'=>'btn btn-lg btn-danger'])}}
    {!! Form::close() !!}
    </div>
```

*SuggestionController.php*

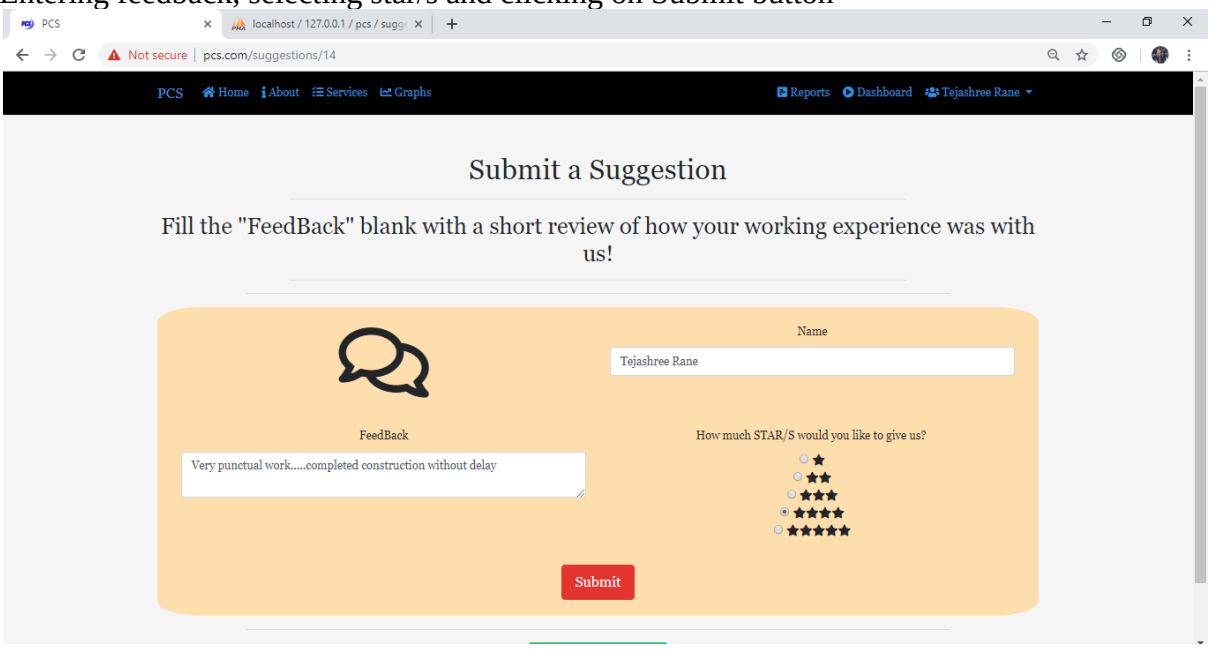
```
public function store(Request $request)
  {
    //validating input
    $this->validate($request,[
        'name'=>'required',
        'content'=>'required',
        'stars'=>'required'
```

```
        ]);
        //creating new object and saving values
        //retrieved from form input
        $sug = new Suggestion;
        $sug->user_id = auth()->user()->id;
        $sug->name = $request->input('name');
        $sug->content = $request->input('content');
        $sug->stars = $request->input('stars');
        $sug->save();
        //redirecting to final page
        return view('suggestions.display');
    }
```

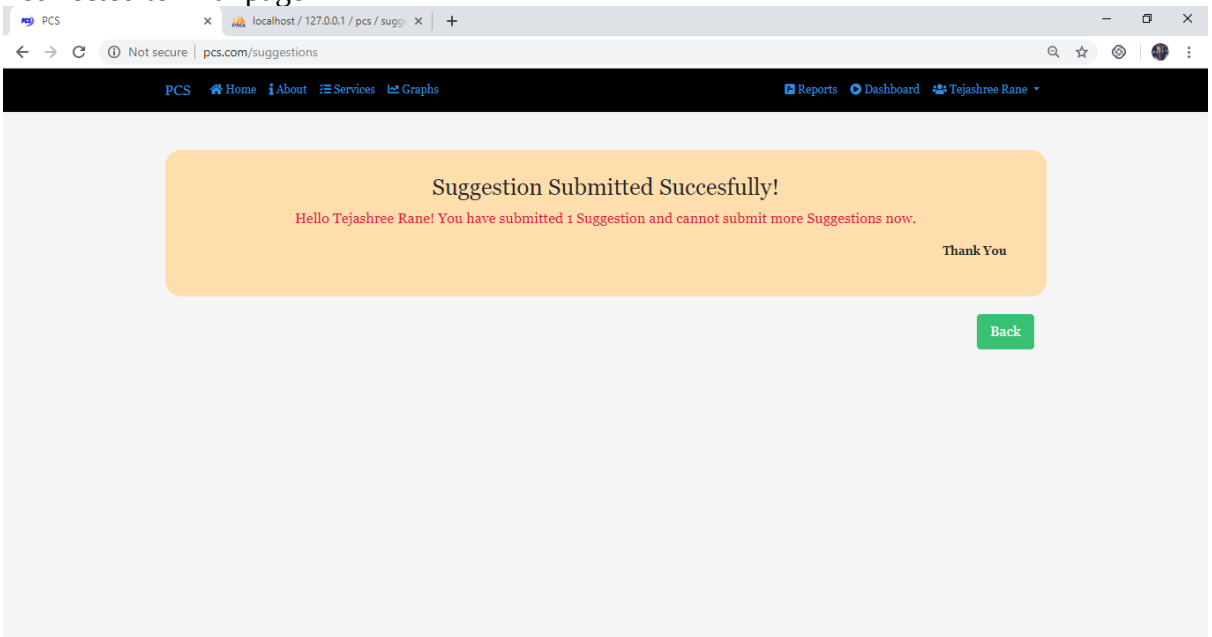
*Suggestions/display.blade.php*

```
<!-- Informing user that he/she has submitted feedback successfully -->
    <div align="center" class="container">
        <div style="border-radius:20px;padding-bottom:30px;padding-top:30px;background-color:navajowhite;">
        <h2>Suggestion Submitted Succesfully!</h2>
        <p style="color:crimson;font-size:18px">Hello {{Auth::user()->name}}! You have submitted 1 Suggestion and cannot submit more Suggestions now.</p>
        <p style="padding-right:50px" align="right"><b>Thank You</b></p>
    </div>
```

Entering feedback, selecting star/s and clicking on Submit button-



Redirected to final page-



## 5.3 Testing Approach

Usually testing begins when the whole software is developed successfully. But here we have followed Iterative model where 1 module is developed and when next module is developed, previous module is redeveloped resulting of a better version than before.

**Software testing** is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product.

Two types of testing performed on this project modules are-

### 5.3.1 Unit Testing

Unit testing involves testing of each and every small module developed one after another. Here modules are tested as and when they are developed. When the new module is developed, new module and the previous module both are tested sequentially. All modules once created are tested and improved before developing next module.

### 5.3.2 Integrated Testing

Integrated Testing is the phase in software testing in which individual software modules are combined and tested as a group. Integration testing is conducted to evaluate the compliance of a system or component with specified functional requirements. Here after successfully completing development of all small modules, all modules are integrated together and hence the testing performed is called as integrated testing. After developing modules of all 3 actors, testing was performed in such a way that all 3 actors are able to get to know what changes/updates etc are done by them.

## 5.4 Modifications and Improvements

After successful unit testing as well as integrated testing, a number of changes in frontend, backend, logic were listed down depending on each other. Since the system needs to be fully bug free and errorless, thus modification in source code is a necessary part before deployment of software.

Some modifications performed in the system after testing-

1. Initially MainAdmin was authorized to search any complaint, after modifying source code MainAdmin can now search and delete a complaint.

2. Initially User was able to submit as many feedbacks as they want i.e. more than 1 feedback to the portal, after modification each User can submit only 1 feedback.

3. Initially SubAdmin was able to only update status by selecting a complaint from a big list, after modifying source code now SubAdmin can search a particular complaint and can update its status as required.

4. Initially User can only submit details of a pothole complaint in fewer textboxes, after modifying source code as well as table structure, User is able to submit all the details in described field for better understanding.

Improvement is a result obtained after modification of source code for frontend or logic in controller or table structure in backend.

## 5.5 Test Cases

A **TEST CASE** is a set of conditions or variables under which a tester will determine whether a system under test satisfies requirements or works correctly.

The process of developing test cases can also help find problems in the requirements or design of an application.

| Test Case ID | Test Scenario | Test Steps | Test Data | Expected Results | Actual Results | Pass/ Fail |
|---|---|---|---|---|---|---|
| | | | | | | |
| TU_01 | Check User Login with valid data. | 1. Go to login page.<br>2. Enter emailID & password.<br>3. Click Login. | emailID=abc@gmail.com, password=1234 | User should login into site. | As expected | Pass |
| | | | | | | |
| TU_02 | Check User submitting a pothole complaint | 1. Go to complaint register page.<br>2. Enter all details of pothole.<br>3.Click Submit | Number of potholes = 3, address=def ghi road jkl society mno mall ghatkopar Mumbai 400086 | User should submit the complaint. | As expected | Pass |
| | | | | | | |
| TU_03 | Check User submitting a suggestion/feedback | 1. Go to Feedback page.<br>2. Enter statement and rate by giving star. | Statement=very punctual working but needs to update status regularly, stars=3 | User should submit the feedback. | As expected | Pass |
| | | | | | | |
| TU_04 | Check SubAdmin updating status of complaint | 1. Go to Complaints list section.<br>2. Search for a complaint.<br>3. Click on status (hyperlinked).<br>4. Select new status from dropdown option.<br>5. Click Update. | Status=Work in Progress | SubAdmin should update the status of complaint. | As expected | Pass |
| | | | | | | |

| Test Case ID | Test Scenario | Test Steps | Test Data | Expected Results | Actual Results | Pass/ Fail |
|---|---|---|---|---|---|---|
| | | | | | | |
| TU_05 | MainAdmin Assigns SubAdmin to a ward. | 1. Go to Assign SubAdmin page.<br>2. Select an account to assign it with a ward.<br>3. Select a ward to be assigned.<br>4. Click Assign. | Account-Name=xyz<br>Emailid=xyz123@gmail.com<br>User_id=1<br>Ward-Wardname=pqrs | MainAdmin should assign account to a ward. | As expected. | Pass |
| | | | | | | |
| TU_06 | MainAdmin dismissing a SubAdmin from a Ward. | 1. Go to Dismiss SubAdmin page.<br>2. Select which account to be dismissed.<br>3. Click on Dismiss. | | MainAdmin should dismiss an account from his/her ward. | As expected. | Pass |
| | | | | | | |
| TU_07 | MainAdmin deleting any user account. | 1. Go to Delete User account page.<br>2. Search for any unwanted/fake account.<br>3. Click on Delete. | | MainAdmin should delete the user account. | As expected. | Pass |
| | | | | | | |
| TU_08 | MainAdmin adding a ward to list. | 1. Go to Add ward page.<br>2. Enter wardname in textbox.<br>3. Click on Add. | Wardname=lmno | MainAdmin should add ward to the list. | As expected. | Pass |
| | | | | | | |
| TU_09 | MainAdmin removing a ward from list. | 1.Go to Remove ward page.<br>2.View the list of ward.<br>3.Click on Remove for which ward wants to be removed. | | MainAdmin should remove the ward from the list. | As expected. | Pass |
| | | | | | | |
| TU_10 | MainAdmin/ SubAdmin searching any complaint. | 1. Go to search section.<br>2. Select filter and enter keyword.<br>3. Click on Search. | Filter=Pincode<br>Keyword=400086 | All complaints are listed to MainAdmin /SubAdmin | As expected | Pass |
| | | | | | | |

# Chapter 6

# Results and Discussion

## 6.1 Test Reports

Well done test report allows us to evaluate the current status of project and quality of the product. There is an ability to take corrective actions if it is necessary. The test report can be the final document which determines if the product is ready for release or not. The test reports created here are based on test cases performed in previous chapter.

| | |
|---|---|
| Test Report ID : | TR_01 |
| Date created : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | User |
| Description : | User logging in using his/her credentials |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_02 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | User |
| Description : | User submitting/registering a pothole complaint with all details |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_03 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | User |
| Description : | User submitting suggestion/feedback of how he/she experienced the service provided by website |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_04 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | SubAdmin |
| Description : | SubAdmin updates status of any complaint of which is in any construction/working phase. |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_05 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | MainAdmin |
| Description : | MainAdmin assigns a SubAdmin to a ward. The assigned SubAdmin will work on complaints submitted under his/her ward. |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_06 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | MainAdmin |
| Description : | MainAdmin dismisses a SubAdmin from a ward. The dismissed SubAdmin will no longer be an authorized person. |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_07 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | MainAdmin |
| Description : | MainAdmin deletes any user account which is fake/duplicate/unwanted/creating trouble by submitting wrong pothole complaints. |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_08 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | MainAdmin |
| Description : | MainAdmin adds a ward for the service to be provided there too. |
| Result : | Successful, but needs to add 1 or more detail with wardname like pincode, etc. |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_09 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | MainAdmin |
| Description : | MainAdmin removes a ward from the list that doesn't needs services, like a ward that doesn't have any complaint/all complaints are solved/etc. |
| Result : | Successful |
| Status: | Pass |

| | |
|---|---|
| Test Report ID : | TR_10 |
| Created at : | 16-03-2019 |
| Created by : | Vicky Gupta |
| Actor : | MainAdmin/SubAdmin |
| Description : | Both highly authorized actors MainAdmin and SubAdmin have a common functionality in their view of searching a complaint to take further action. MainAdmin can search a complaint and have authority to delete if necessary. SubAdmin can search a complaint and have authority to update the status of it. |
| Result : | Successful |
| Status: | Pass |

## 6.2 User Documentation

How the software works:-

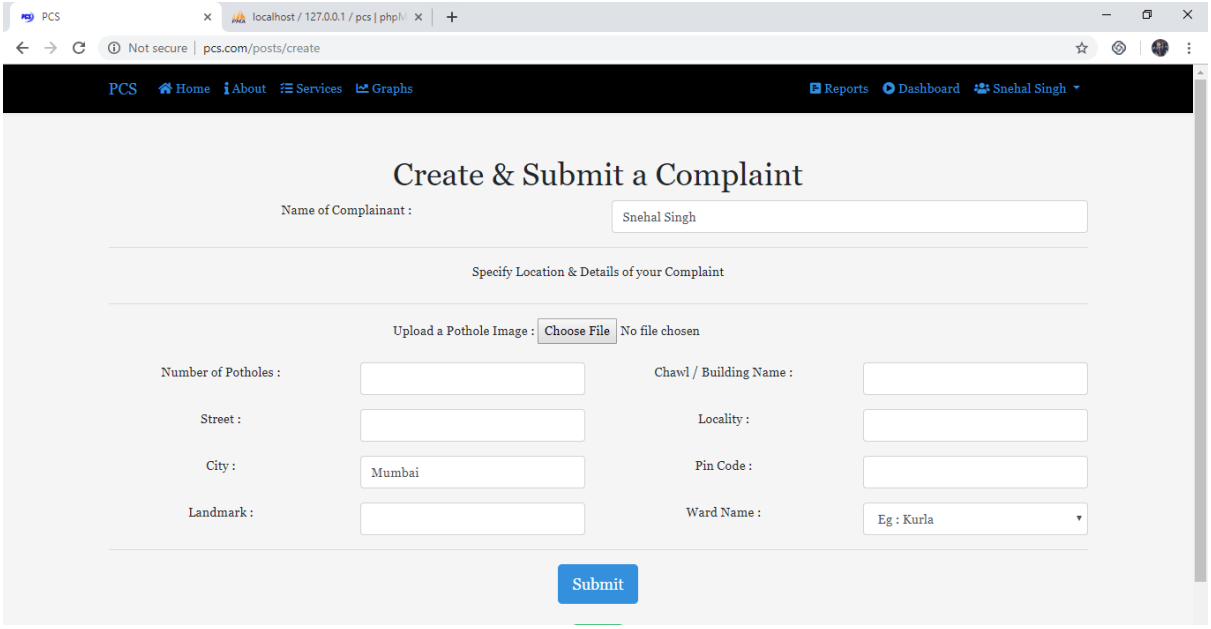
3 actors are involved in the system namely User, SubAdmin and MainAdmin. All of them are authorized with various different as well as some common functionality to execute.

The main flow of the system is that a user submits a pothole complaint and the respective SubAdmin manages the complaint taking it to 4 phases i.e. Seen, Beginning Work, Work in Progress and Finished Work. A complaint when submitted has a status of New by default.

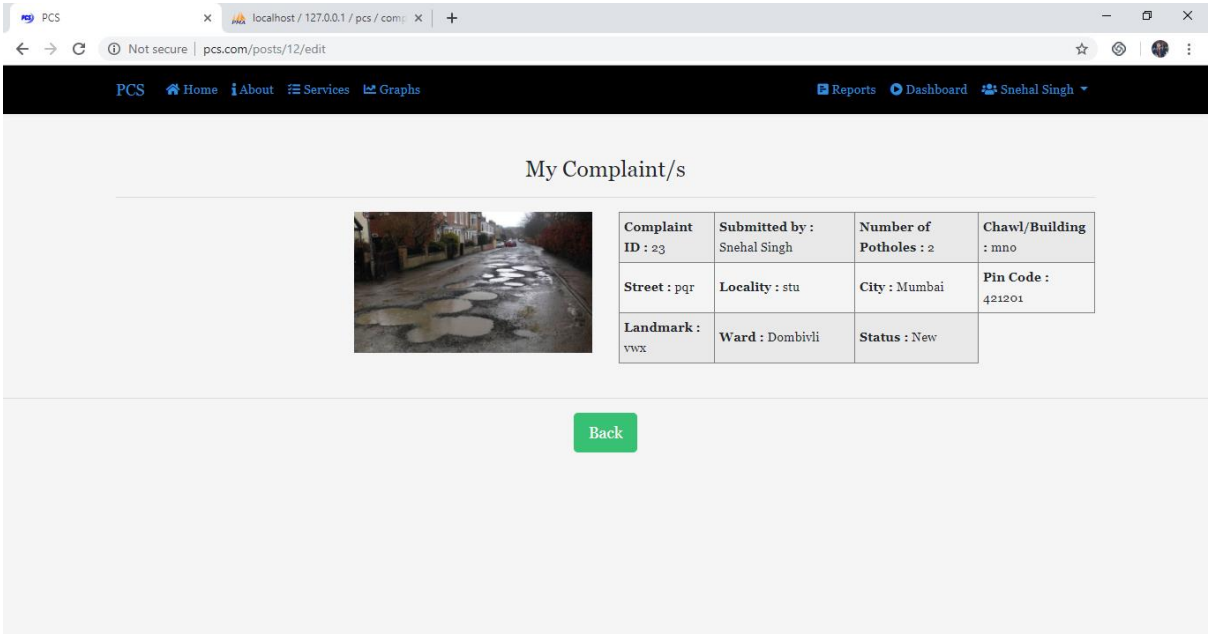
Rest all functions or authorities given to actors are based on checking some information/managing system/etc. Like a user has functions of viewing registered complaints nearby his/her ward, viewing all complaints registered till date, checking updated status of their submitted complaint, submitting a feedback/suggestion after using the service provided by the website. Also SubAdmin other than having authority of updating status of complaint has authority to search any particular complaint for further action. And the highly authorized actor MainAdmin has a number of functionalities to execute i.e. assigning/dismissing a SubAdmin to/from a ward, viewing all complaints and deleting any complaint, searching and deleting a complaint, adding/removing ward according to service requirements, deleting any fake/duplicate/unwanted user account, viewing all suggestions/feedback for analyzing how the services is to be improved.

**Screenshots of all 3 actors with their authorized functionalities-**

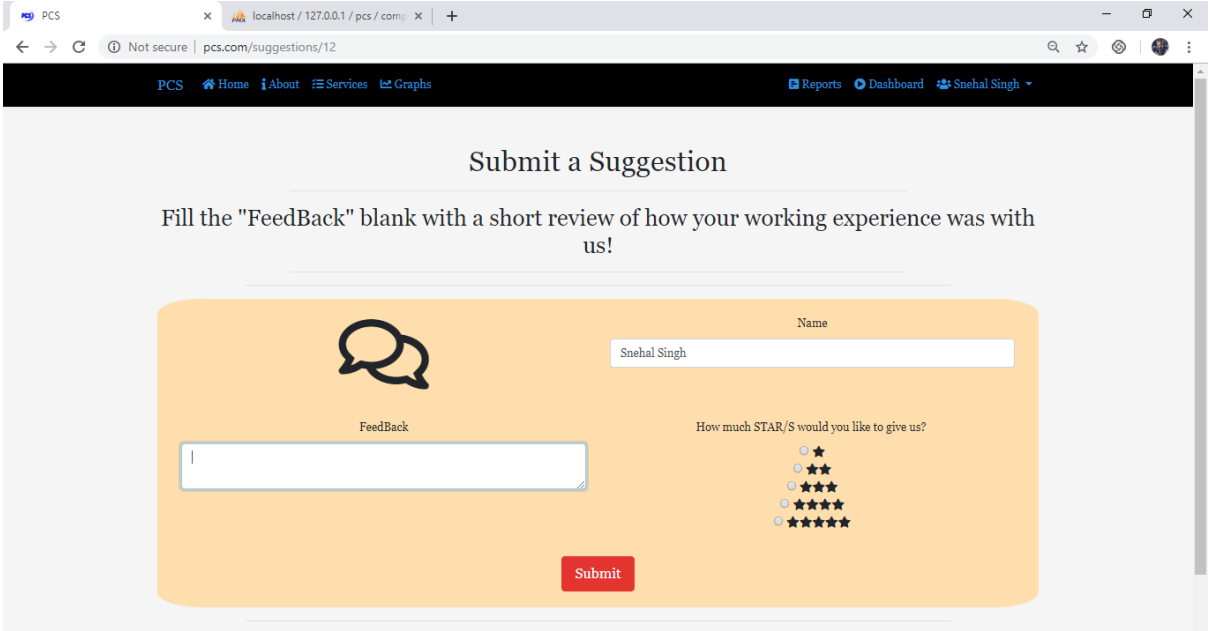
User Submitting a Pothole Complaint-



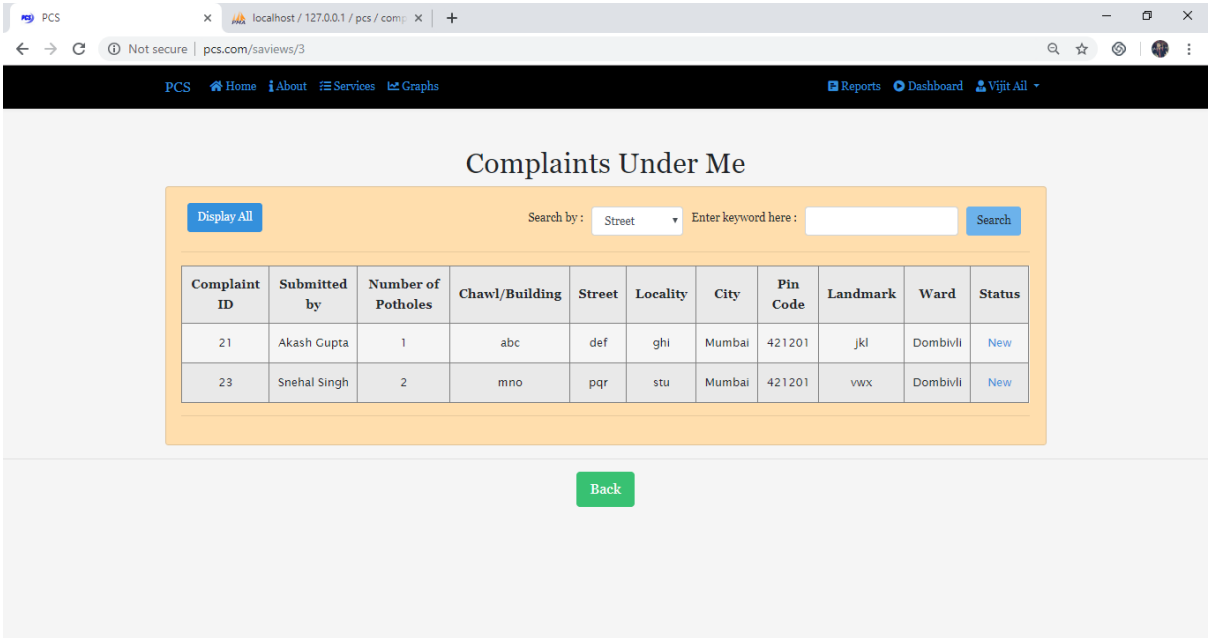

User viewing their submitted complaint's status-



User submitting a feedback/suggestion-

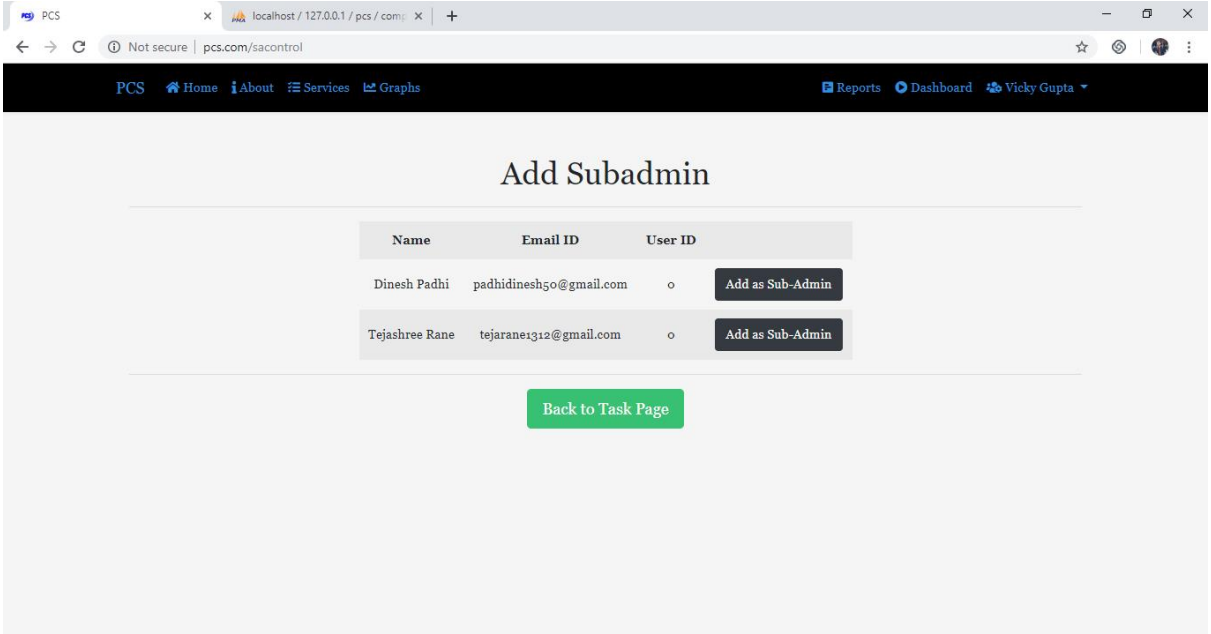



SubAdmin viewing complaints under his/her assigned ward to update status-

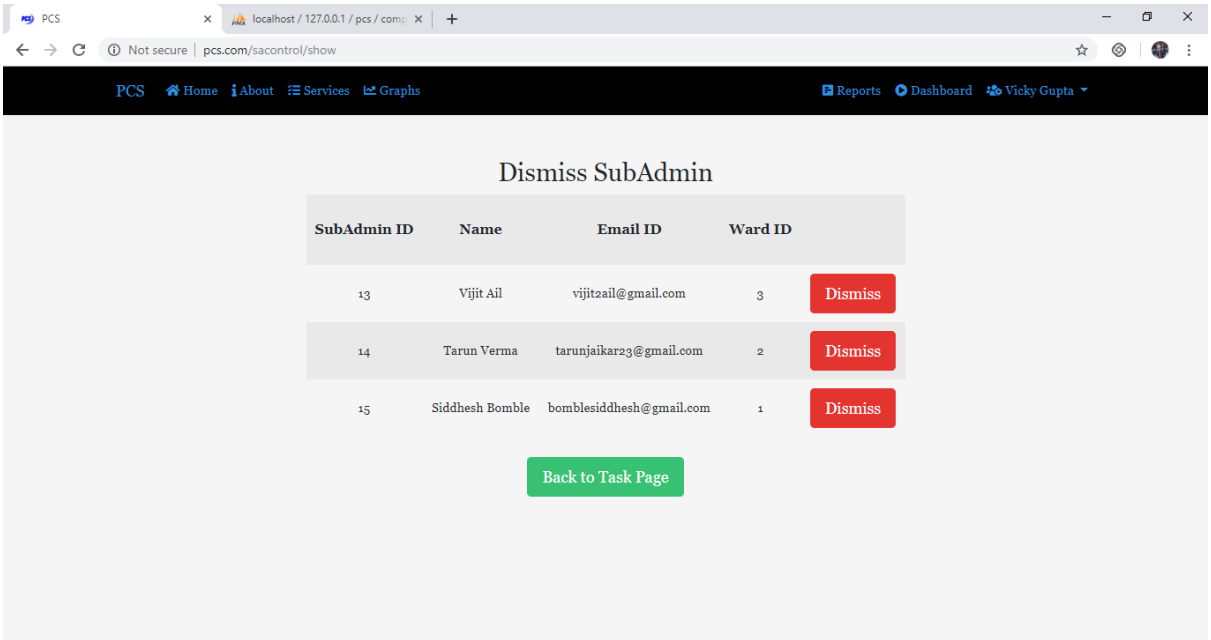



| Complaint ID | Submitted by | Number of Potholes | Chawl/Building | Street | Locality | City | Pin Code | Landmark | Ward | Status |
|---|---|---|---|---|---|---|---|---|---|---|
| 21 | Akash Gupta | 1 | abc | def | ghi | Mumbai | 421201 | jkl | Dombivli | New |
| 23 | Snehal Singh | 2 | mno | pqr | stu | Mumbai | 421201 | vwx | Dombivli | New |

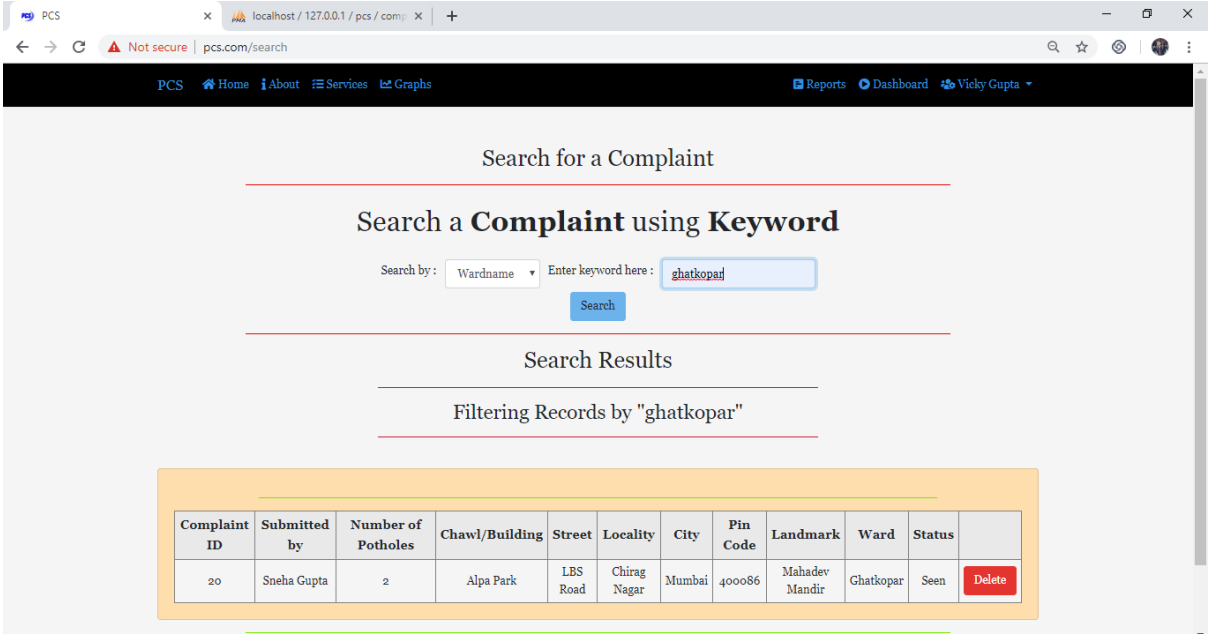
MainAdmin viewing lists of accounts from which to be assigned with any ward-



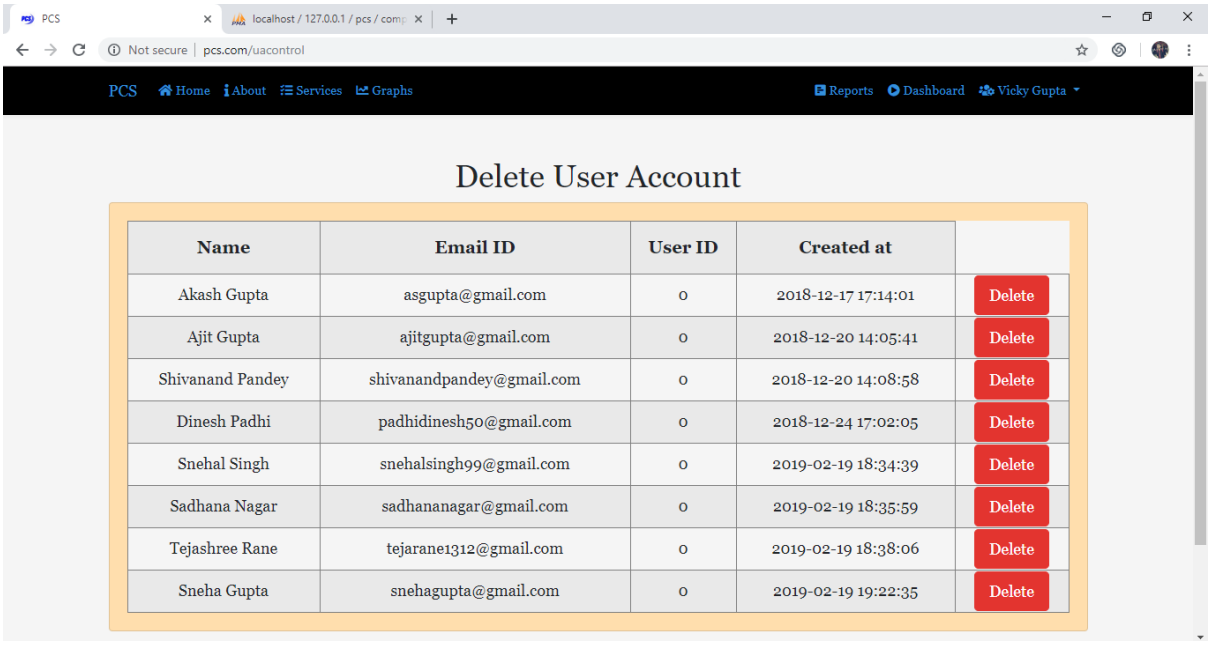

MainAdmin viewing lists of SubAdmin's to dismiss if not required-



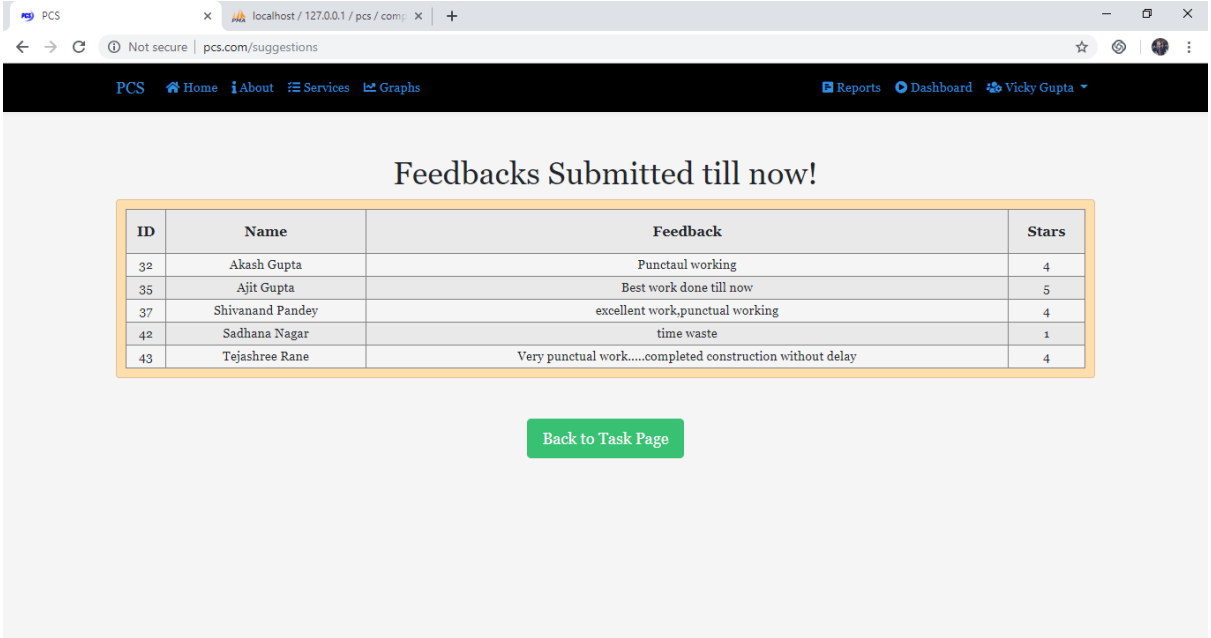
MainAdmin searching for a complaint-




MainAdmin deleting any user account-



MainAdmin viewing all suggestions/feedback submitted by users-




| ID | Name | Feedback | Stars |
|---|---|---|---|
| 32 | Akash Gupta | Punctaul working | 4 |
| 35 | Ajit Gupta | Best work done till now | 5 |
| 37 | Shivanand Pandey | excellent work,punctual working | 4 |
| 42 | Sadhana Nagar | time waste | 1 |
| 43 | Tejashree Rane | Very punctual work.....completed construction without delay | 4 |

# Chapter 7

# Conclusions

## 7.1 Conclusion

As due to increasing potholes in city and guiltless websites which do not cooperate with citizens or don't take action on complaints, the system built here is to overcome the problems and provide solution for Pothole Complaints.

The system built here is capable of providing easy functionalities that each user can easily understand & use in first try. There are still some functionalities that are to be developed later for better service.

The system developed is capable of facing majority of problems. As required the modules and functions can be changed/removed/developed when there is a need.

## 7.2 Limitations of the system

Limitations here describe what situations the system cannot handle or which module/functionality has not yet been developed. Some limitations are that if the user is not satisfied when the pothole is resolved the user cannot request to reconstruct the pothole or cannot request to restart the complaint. Even after providing all fields of address for submitting a pothole complaint exact location of pothole cannot be determined because of absence of longitude and latitude detection of pothole via image.

Some other limitations will be determined when there is a development of new module later or when the system can be used officially.

## 7.3 Future Scope of the Project

If the functionalities developed are understandable and uncomplicated for the User, SubAdmin and MainAdmin then the system can be used at a good level (only after developing more modules).

Scope of the Project currently is at a better level than expected as the project is made by students i.e. made at student level of understanding and student level skills. After developing and adding more functions & by increasing the services more and more by interacting with Users suggestions/feedback the Scope of the Project will be increased.